



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

UNIVERSIDAD POLITÉCNICA DE VALENCIA

Escuela Técnica Superior de Ingeniería Informática

Departamento de Sistemas Informáticos y Computación

Trabajo Fin de Máster

Arquitectura multiagente para la toma de decisiones en mercados de activos digitales

Máster Universitario en Inteligencia Artificial, Reconocimiento de Formas
e Imagen Digital

Autor

Shiyi Cheng

Tutora

Eva Onaindia de la Rivaherrera

Curso 2025–2026

Resumen

Los mercados de activos digitales mueven cientos de millones anuales y, aun así, carecen de herramientas analíticas y de automatizaciones integradas o de libre uso. La motivación principal de este proyecto nace de esta profunda brecha tecnológica. Actualmente, la gestión de carteras en la mayoría de los mercados se realiza de forma manual o mediante herramientas estáticas, lo que resulta altamente ineficiente y arriesgado.

El propósito principal de este TFM es evaluar si un ecosistema descentralizado de agentes de inteligencia artificial puede superar las limitaciones humanas y generar rendimientos estables en mercados de alta fricción. Se opta por una arquitectura multiagente de aprendizaje por refuerzo para descentralizar la toma de decisiones y reducir la complejidad computacional del mercado. El objetivo es que los agentes aprendan estrategias de inversión óptimas que maximicen el saldo total de la cartera.

La metodología desarrollada en el TFM se aplica al ámbito de la economía virtual de Counter-Strike 2, un popular videojuego que cuenta con un ecosistema financiero integrado con una alta frecuencia de intercambios.

Palabras clave: Mercados de activos digitales, toma de decisiones, arquitectura multiagente, aprendizaje por refuerzo.

Resum

Els mercats d'actius digitals mouen centenars de milions anuals i, tot i això, encara no disposen d'eines analítiques ni d'automatitzacions integrades o de lliure ús. La motivació principal d'aquest projecte naix d'aquesta profunda bretxa tecnològica. Actualment, la gestió de carteres en la majoria dels mercats es realitza de forma manual o mitjançant eines estàtiques, la qual cosa resulta altament ineficient i arriscada.

El propòsit principal d'aquest TFM és avaluar si un ecosistema descentralitzat d'agents d'intel·ligència artificial pot superar les limitacions humanes i generar rendiments estables en mercats d'alta fricció. S'opta per una arquitectura multiagent d'aprenentatge per reforç per descentralitzar la presa de decisions i reduir la complexitat computacional del mercat. L'objectiu és que els agents aprenguen estratègies d'inversió òptimes que maximitzen el saldo total de la cartera.

La metodologia desenvolupada en el TFM s'aplica a l'àmbit de l'economia virtual de Counter-Strike 2, un videojoc popular que compta amb un ecosistema financer integrat d'alta freqüència i liquiditat.

Paraules clau: Mercats d'actius digitals, presa de decisions, arquitectura multiagent, aprenentatge per reforç.

Abstract

Digital asset markets move hundreds of millions annually and yet still lack integrated or freely available analytical tools and automation systems. The main motivation of this project stems from this deep technological gap. At present, portfolio management in most markets is carried out manually or through static tools, which makes it highly inefficient and risky.

The main purpose of this master's thesis is to evaluate whether a decentralized ecosystem of artificial intelligence agents can overcome human limitations and generate stable returns in high-friction markets. A multi-agent reinforcement learning architecture is chosen to decentralize decision-making and reduce the computational complexity of the market. The goal is for the agents to learn optimal investment strategies that maximize the total portfolio balance.

The methodology developed in this master's thesis is applied to the virtual economy of Counter-Strike 2, a popular video game with an integrated financial ecosystem characterized by high frequency and liquidity.

Key words: Digital asset markets, decision-making, multi-agent architecture, reinforcement learning.

Índice general

Índice de figuras	6
Índice de tablas	7
Lista de algoritmos	8
Siglas y abreviaturas	9
Glosario	11
1 Introducción	15
1.1 Motivación	15
1.2 Planteamiento del problema	16
1.3 Objetivos	16
1.4 Estructura del documento	17
2 Estado del arte	18
2.1 Mercados y plataformas de activos digitales	18
2.2 Aprendizaje supervisado y no supervisado	20
2.3 Aprendizaje por refuerzo	21
2.4 Aprendizaje por refuerzo multiagente	23
2.5 Gestión de carteras y riesgo	25
2.6 Aplicaciones y trabajos relacionados	26
3 Contextualización del caso de estudio: activos digitales en Counter-Strike 2	28
3.1 Justificación y alcance	28
3.2 Artículos y atributos de Counter-Strike 2	29
3.3 Mercados secundarios y condiciones de intercambio	30
4 Modelo de decisión y arquitectura multiagente propuesta	32
4.1 Problema de decisión del caso de estudio	32
4.2 Modelo MARL aplicado al caso de estudio	34
4.3 Arquitectura del sistema	37
4.4 Agentes de decisión	39
4.5 Función de recompensa	40
4.6 Restricciones de riesgo y validez de acciones	45
5 Entorno y preparación de datos	47
5.1 Fuentes de datos	47
5.2 Normalización y validación de los datos	48
5.3 Variables y conjunto temporal	48
6 Implementación y operación	50
6.1 Componentes y responsabilidades	50
6.2 Adquisición y persistencia de datos	51
6.3 Entrenamiento y ejecución de modelos	53
6.4 Consola local de consulta	54

7	Experimentación y análisis	55
7.1	Diseño experimental	55
7.2	Validación técnica y pruebas	56
7.3	Métricas	57
7.4	Resultados	59
7.4.1	Migración de reglas económicas	59
7.4.2	Resultados del modelo supervisado	59
7.4.3	Evolución de los datasets de trading	60
7.4.4	Conjunto actual de compraventa	61
7.4.5	Exploración supervisada de marzo	61
7.4.6	Ablación de histórico y aumentación	64
7.4.7	Comprobación del corte reciente	65
7.4.8	Evidencia de pruebas automatizadas	66
7.4.9	Validación funcional de OCR	66
7.4.10	Uso operativo de los datos de compraventa	67
7.4.11	Dificultades y pivotaciones	67
7.4.12	Controles funcionales del entorno MARL	68
7.4.13	Resultados de los controles funcionales	68
7.4.14	Matriz de entrenamiento MARL	69
7.5	Evaluación final reservada	71
7.6	Discusión	71
7.6.1	Amenazas a la validez	71
7.6.2	Casos favorables	71
7.6.3	Limitaciones observadas	72
7.6.4	Lectura de los resultados	72
8	Conclusiones	73
8.1	Conclusiones principales	73
8.2	Cumplimiento de objetivos	74
8.3	Limitaciones	74
8.4	Trabajo futuro	74
	Bibliografía	76
A	Anexos	79
A.1	Estructura del repositorio	79
A.2	Configuración del entorno	80
A.3	Comandos de datasets y modelos	80
A.4	Parámetros de entrenamiento	80
A.5	Modelo de datos	80
A.6	Resultados adicionales	81

Índice de figuras

2.1	Ciclo de interacción entre un agente y un entorno en aprendizaje por refuerzo.	22
2.2	Interacción general en MARL con entrenamiento centralizado y ejecución descentralizada.	25
4.1	Niveles principales de la arquitectura propuesta.	37
4.2	Construcción del estado y retroalimentación de la decisión simulada. . .	38
7.1	Evolución de cortes.	60
7.2	Tamaño y distribución de la etiqueta en <code>trading_profit_v2</code>	61
7.3	Corte temporal de marzo.	62
7.4	Modelos en marzo.	63
7.5	Histórico y aumentación.	64
7.6	Métricas por umbral.	65
7.7	Resultados de validación de la matriz MARL. Cada punto representa una semilla.	70
A.1	Estructura del repositorio.	79

Índice de tablas

2.1	Ejemplos de mercados integrados.	20
2.2	Ejemplos de plataformas externas.	20
7.1	Cobertura de pruebas.	57
7.2	Umbrales de dirección.	59
7.3	Cortes de datos.	60
7.4	Particiones temporales de <code>trading_profit_v2</code>	61
7.5	Configuraciones de marzo.	63
7.6	Comparativa de marzo.	63
7.7	Ablación logística.	64
7.8	Corte BUFF–Steam.	65
7.9	Pruebas automatizadas.	66
7.10	Validación funcional de OCR.	66
7.11	Episodio MARL.	68
7.12	Ejecuciones MARL.	69
7.13	Resultados de validación de la matriz MARL.	70
A.1	Configuración experimental.	81

Lista de algoritmos

1	Flujo de información y decisión simulada	39
2	Decisión cooperativa	40
3	Flujo de scraping.	51
4	Entrenamiento con PPO	56

Siglas y abreviaturas

Esta relación recoge únicamente las siglas, códigos y abreviaturas que se emplean de forma recurrente.

API Interfaz que permite que dos programas intercambien datos o funciones.

BUFF Plataforma externa de compraventa de activos digitales, usada como fuente de precios en el caso de estudio.

CLAHE Técnica de mejora de contraste de una imagen por zonas, del inglés *Contrast Limited Adaptive Histogram Equalization*.

CLI Interfaz de línea de comandos, del inglés *Command-Line Interface*.

CNY Código internacional del yuan chino.

CS2 *Counter-Strike 2*, videojuego empleado como caso de estudio.

CSV Formato de datos tabulares separado por comas, del inglés *Comma-Separated Values*.

CTCE Entrenamiento centralizado y ejecución centralizada, del inglés *Centralized Training with Centralized Execution*.

CTDE Entrenamiento centralizado y ejecución descentralizada, del inglés *Centralized Training with Decentralized Execution*.

EUR Código internacional del euro.

F1 Métrica que combina precisión y exhaustividad mediante su media armónica.

IA Inteligencia artificial.

MADDPG Algoritmo de aprendizaje por refuerzo multiagente, del inglés *Multi-Agent Deep Deterministic Policy Gradient*.

MAPPO Versión multiagente de PPO, del inglés *Multi-Agent Proximal Policy Optimization*.

MARL Aprendizaje por refuerzo multiagente, del inglés *Multi-Agent Reinforcement Learning*.

MDP Modelo matemático de decisión secuencial, del inglés *Markov Decision Process*.

OCR Reconocimiento de texto en imágenes, del inglés *Optical Character Recognition*.

PPO Algoritmo de optimización de políticas, del inglés *Proximal Policy Optimization*.

POMDP Proceso de decisión de Markov parcialmente observable, del inglés *Partially Observable Markov Decision Process*.

PR-AUC Área bajo la curva precisión-exhaustividad, del inglés *Precision-Recall Area Under the Curve*.

RL Aprendizaje por refuerzo, del inglés *Reinforcement Learning*.

RLlib Biblioteca de Ray utilizada para entrenar y evaluar políticas de aprendizaje por refuerzo.

ROC-AUC Área bajo la curva ROC, del inglés *Receiver Operating Characteristic Area Under the Curve*.

RSI Indicador de la intensidad de los cambios de precio, del inglés *Relative Strength Index*.

TFM Trabajo Fin de Máster.

Glosario

Este glosario reúne los términos de mercado y decisión empleados de forma recurrente en la memoria.

Activo digital Elemento asociado a una cuenta de usuario o a un servicio digital que puede utilizarse, coleccionarse o intercambiarse.

Acción conjunta Conjunto formado por las acciones que todos los agentes de un entorno multiagente toman en un mismo instante.

Agente Programa que toma decisiones a partir de la información que recibe.

Aprendizaje por refuerzo Método de aprendizaje en el que un agente observa un entorno, selecciona acciones y recibe recompensas que permiten evaluar sus consecuencias.

Aprendizaje por refuerzo multiagente Extensión del aprendizaje por refuerzo en la que dos o más agentes interactúan con un mismo entorno y pueden cooperar, competir o decidir de forma independiente.

Beneficio neto Resultado económico de una operación después de descontar el precio de compra, las comisiones y los costes aplicables.

Brier score Métrica que evalúa el error de una probabilidad estimada comparándola con el resultado observado.

Calibración de probabilidades Comprobación de si las probabilidades producidas por un modelo coinciden con la frecuencia real observada.

Capital bloqueado Parte del saldo que permanece comprometida en una compra o que no puede reutilizarse mientras el activo no pueda venderse o transferirse.

Cartera Conjunto formado por el saldo disponible y las posiciones abiertas, es decir, los activos digitales adquiridos que aún no se han vendido.

Checkpoint Modelo guardado durante o después de un entrenamiento para poder evaluarlo, reutilizarlo o continuar el proceso más adelante.

Cookie de sesión Dato local generado por una plataforma web que permite reconocer una sesión iniciada sin volver a introducir las credenciales en cada petición.

Demanda Número de personas usuarias que quieren adquirir un activo digital.

Disponibilidad Número de unidades de un activo digital que están a la venta en una plataforma.

Drawdown Mayor caída del valor de una cartera respecto a un máximo anterior durante un periodo de evaluación.

Economía de videojuego Conjunto de reglas con las que un videojuego permite obtener, poseer, utilizar e intercambiar activos digitales.

Economía persistente Economía de videojuego en la que los activos digitales permanecen asociados a la cuenta de la persona usuaria después de terminar una partida.

Entorno Representación de la situación en la que un agente toma decisiones, formada por la información disponible y las reglas que determinan las consecuencias de sus acciones.

Episodio Ejecución completa de un entorno de aprendizaje por refuerzo, desde su estado inicial hasta la condición que marca su finalización.

Escasez Número de unidades existentes de un activo digital en todo el mundo.

Estado global Información completa que el entorno utiliza en un instante para calcular observaciones, transiciones y recompensas.

Exposición Parte de la cartera comprometida con un activo, una plataforma o una categoría de activos.

Exhaustividad Métrica de clasificación, también llamada *recall*, que mide qué proporción de los casos positivos reales detecta un modelo.

Función de transición Regla del entorno que determina cómo cambia el estado después de ejecutar una acción.

Horizonte temporal Periodo previsto entre la compra de un activo y su posible venta.

Mercado cerrado Entorno en el que los activos solo se utilizan dentro del videojuego y no se intercambian entre personas usuarias.

Mercado centralizado Mercado en el que una empresa o plataforma administra las cuentas, los activos digitales o las reglas de intercambio.

Mercado descentralizado Entorno en el que la gestión del activo y las reglas de intercambio no dependen de una plataforma central.

Mercado digital Entorno en línea en el que las personas usuarias intercambian bienes o servicios mediante plataformas.

Mercado integrado Mercado administrado por el propio ecosistema del videojuego o de distribución, que controla los activos digitales asociados a las cuentas y las reglas de intercambio.

Mercado secundario Mercado de reventa entre personas usuarias.

Mypy Herramienta de comprobación estática de tipos para Python que revisa si el uso de variables y funciones es coherente con sus anotaciones.

Oferta Publicación mediante la cual una persona pone a la venta un activo digital identificado por sus características relevantes e indica su precio.

Operación Decisión sobre un activo digital. Puede consistir en comprar, vender, mantener una posición abierta o descartar una compra o una venta.

Operación de compraventa Análisis conjunto de una compra de un activo digital y su venta posterior.

Oportunidad Posible operación de compraventa que merece revisarse porque sus datos iniciales indican que podría generar un beneficio neto.

Observación local Parte del estado global que recibe un agente concreto para tomar su decisión.

Observabilidad parcial Situación en la que un agente no recibe el estado completo del entorno, sino una observación limitada de la información disponible.

Orden de compra Publicación que indica el precio máximo que una persona está dispuesta a pagar por un activo digital concreto.

Plataforma Servicio digital donde se administran las reglas del intercambio y la información necesaria para participar en él.

Plataforma externa Plataforma distinta del servicio que administra originalmente los activos digitales vinculados a la cuenta y que publica ofertas o facilita operaciones sobre activos transferibles.

Política Regla o modelo que indica qué acción selecciona un agente a partir de la información que observa.

Posición abierta Activo digital ya adquirido que permanece en la cartera hasta su venta. Mientras la posición está abierta, el importe empleado en la compra no puede destinarse a otra operación.

Proceso de decisión de Markov parcialmente observable Modelo de decisión secuencial en el que el entorno mantiene un estado, pero el agente solo recibe observaciones parciales de ese estado.

Pytest Herramienta de pruebas para Python que localiza funciones de prueba, las ejecuta y comunica si el comportamiento observado coincide con el esperado.

Rareza Dificultad para obtener un activo digital.

Recompensa Señal numérica que evalúa la consecuencia de una acción en aprendizaje por refuerzo y guía el aprendizaje de una política.

Rentabilidad Relación entre el beneficio neto de una operación y el capital empleado para realizarla.

Rendimiento técnico Medida del coste práctico de ejecutar un sistema, como tiempo de proceso, volumen de datos tratado o recursos utilizados.

Restricción de riesgo Regla que limita una operación para evitar que la cartera concentre demasiado capital, pierda liquidez o incumpla condiciones operativas.

Ruff Herramienta para Python que revisa reglas de estilo, errores estáticos y formato de código.

Señal supervisada Valor producido por un modelo entrenado con ejemplos históricos y utilizado como información adicional antes de tomar una decisión.

Scraping Proceso de consulta automática de una fuente web para extraer datos y convertirlos en registros estructurados.

Spread Diferencia entre el mejor precio de venta y la mejor orden de compra disponibles para un mismo activo en un momento dado.

Trade hold Bloqueo temporal que impide vender o transferir un activo digital recién adquirido.

Volumen de intercambios Cantidad de unidades de un activo digital vendidas durante un periodo, como un día, una semana o un mes.

Introducción

Los **mercados digitales** son entornos en línea en los que las personas usuarias intercambian bienes o servicios mediante **plataformas**. Una plataforma administra las reglas del intercambio y la información necesaria para participar en él. Cuando los bienes están asociados a una cuenta o a un servicio digital y pueden utilizarse, coleccionarse o intercambiarse, se denominan **activos digitales**.

La gestión de activos digitales plantea decisiones sucesivas de compra, venta, mantenimiento o descarte. Estas decisiones se denominan **operaciones**. Cada operación puede modificar la **cartera**, formada por el saldo disponible y las **posiciones abiertas**, es decir, los activos digitales adquiridos que aún no se han vendido. Por ello, el objetivo no es solo identificar una diferencia de precios, sino evaluar decisiones que afectan a la evolución de la cartera con el fin de generar un beneficio económico.

El **aprendizaje por refuerzo** (RL), del inglés *Reinforcement Learning*, es un paradigma para abordar problemas de decisión secuencial. En este marco, un **agente** interactúa con un **entorno**, selecciona acciones y recibe **recompensas**. A diferencia de un modelo que solo estima un resultado futuro, RL permite estudiar cómo una decisión actual condiciona las decisiones posteriores.

El **aprendizaje por refuerzo multiagente** (MARL), del inglés *Multi-Agent Reinforcement Learning*, amplía este marco cuando varios agentes interactúan con el mismo entorno. Los agentes pueden cooperar, competir o actuar de forma independiente. La coordinación puede ser centralizada, cuando una única entidad toma la decisión conjunta, o descentralizada, cuando cada agente decide desde su propia información y responsabilidad.

1.1 Motivación

Los mercados de activos digitales en videojuegos permiten que las personas usuarias compren, vendan e intercambien objetos asociados a un videojuego. El valor de estos activos digitales puede depender de su utilidad (por ejemplo, aumenta la vida de un personaje), apariencia, **rareza** (la dificultad para obtener un activo digital), **disponibilidad** (el número de unidades en venta) y **demanda** (el número de personas que quieren adquirirlo). La literatura sobre bienes virtuales identifica dimensiones funcionales, estéticas y sociales como determinantes de ese valor [1]. Por ello, una misma operación puede presentar un resultado económico diferente según el momento y la plataforma en los que se realice.

La motivación principal de este trabajo es apoyar la identificación de operaciones que puedan generar un **beneficio neto** (el resultado después de descontar el precio de compra, las comisiones y otros costes). Una decisión acertada puede aportar un resultado económico favorable a la persona usuaria. Sin embargo, ese resultado no está

garantizado y comparar dos precios no es suficiente para decidir. Un mismo activo digital puede aparecer en varias plataformas con precios, divisas, comisiones y condiciones de intercambio distintas. La decisión exige valorar cuánto se pagaría al comprar, cuánto se recibiría al vender y si se aplica un *trade hold* (bloqueo temporal que impide vender o transferir el activo adquirido).

1.2 Planteamiento del problema

Las decisiones de compra y venta se suceden en el tiempo. Cada compra utiliza una parte del saldo disponible de la cartera digital. Mientras el activo digital adquirido permanezca en la cartera y no se pueda vender, ese importe no puede utilizarse para adquirir otro activo digital. Por ello, el problema no consiste solo en comparar precios, sino también en decidir cómo emplear un saldo limitado entre distintas posibilidades de compra.

Las plataformas no siempre emplean la misma moneda, el mismo nombre ni la misma fecha de observación para un activo digital. Antes de comparar precios, es necesario unificar esa información.

Cuando se analizan conjuntamente una compra y una venta posterior, se habla de una **operación de compraventa**. El punto de partida es un procedimiento manual en el que, para cada activo digital, se anotan el precio de compra, el precio de venta, la conversión de moneda, las comisiones, la fecha de desbloqueo y el **capital bloqueado** (el saldo que no puede utilizarse para nuevas compras). Esta primera implementación manual de donde partimos permite evaluar casos concretos, pero obliga a repetir los cálculos y a comprobar los datos de cada activo digital.

Con la información mencionada anteriormente, el sistema debe decidir qué operaciones conviene realizar, mantener o descartar. Para justificar esa decisión, también debe considerar el **volumen de intercambios** (la cantidad de unidades vendidas durante un día, una semana o un mes), las reglas económicas y de intercambio aplicables y el estado de la cartera. La pregunta que debe responder es si la posible operación merece ser estudiada con criterios consistentes.

1.3 Objetivos

El objetivo principal de este TFM es diseñar y evaluar, mediante simulación, una arquitectura multiagente que apoye la identificación y evaluación de operaciones orientadas a generar un beneficio neto en mercados de activos digitales. El caso de estudio es la economía virtual del videojuego *Counter-Strike 2*.

Los objetivos específicos se agrupan en cuatro bloques:

- **Análisis del caso de estudio:** analizar el mercado de activos digitales de *Counter-Strike 2*, sus plataformas, restricciones, comisiones, divisas, volumen de intercambios y efectos del *trade hold*.
- **Construcción de datos:** construir un flujo de adquisición y preparación de datos procedentes de diferentes plataformas de mercado y fuentes manuales, manteniendo la trazabilidad de la fuente, la fecha, la moneda y la calidad del dato.

- **Simulación y estimación:** desarrollar un simulador que incorpore las reglas económicas del procedimiento manual, construir conjuntos de datos temporales y obtener estimaciones de beneficio que puedan incorporarse al entorno de decisión.
- **Coordinación multiagente:** formular un entorno de aprendizaje por refuerzo con el estado de la cartera, las acciones y las recompensas definidos. Sobre esa base, distribuir los roles de detección, decisión y control de riesgo entre agentes especializados y evaluar su funcionamiento con métricas de **rentabilidad**, riesgo, **exposición** y capital bloqueado.

1.4 Estructura del documento

La memoria se estructura de forma secuencial siguiendo las fases principales del proyecto. A continuación, se detalla el contenido de cada capítulo:

- **Capítulo 1:** introduce el contexto del proyecto, la motivación, el problema abordado, la solución propuesta, los objetivos, el alcance y la estructura.
- **Capítulo 2:** presenta el estado del arte sobre mercados de activos digitales, plataformas de intercambio, aprendizaje por refuerzo, coordinación multiagente y toma de decisiones en carteras.
- **Capítulo 3:** analiza los conceptos operativos del mercado de activos digitales en videojuegos y justifica el uso de *Counter-Strike 2* como caso de estudio.
- **Capítulo 4:** concreta el modelo de decisión multiagente aplicado al caso de estudio y presenta la arquitectura propuesta. Se describen el estado, las observaciones, las acciones, la transición simulada, los agentes, la función de recompensa y las restricciones de riesgo.
- **Capítulo 5:** expone el entorno de datos y simulación. Se detallan fuentes de adquisición, persistencia, características utilizadas, simulador de cartera, OCR e interfaz de consulta.
- **Capítulo 6:** describe la implementación y operación del sistema. Presenta la organización del repositorio, el scraping, la persistencia, la consola local y la configuración reproducible.
- **Capítulo 7:** recoge la experimentación realizada. Incluye migración de reglas económicas, construcción de datasets supervisados, evaluación de modelos tabulares, pruebas técnicas y de rendimiento, dataset de trading, estado del entrenamiento MARL, dificultades y pivotaciones detectadas.
- **Capítulo 8:** presenta conclusiones principales, revisa el cumplimiento de objetivos y plantea líneas de trabajo necesarias para completar la validación experimental.
- **Anexos:** reúnen información complementaria sobre estructura del repositorio, configuración del entorno, comandos de ejecución, parámetros de entrenamiento y modelo de datos.

Estado del arte

Este capítulo presenta los fundamentos teóricos y los trabajos previos que contextualizan este trabajo. En primer lugar, sitúa los mercados digitales de activos dentro de las economías de los videojuegos y diferencia los principales tipos de mercado y plataforma. Esta distinción permite entender que las posibilidades de intercambio, los precios observados y las reglas de cada operación dependen del entorno en el que se realiza.

Después, el capítulo introduce el aprendizaje supervisado como método para obtener estimaciones a partir de datos históricos. A continuación, presenta los fundamentos del aprendizaje por refuerzo en el caso de un único agente, incluyendo la interacción entre agente y entorno, los procesos de decisión de Markov, las políticas, las recompensas y el retorno acumulado. Estos conceptos se amplían después al aprendizaje por refuerzo multiagente, donde varios agentes actúan sobre un entorno compartido. Se revisan las formas de cooperación, las recompensas individuales y colectivas, y los enfoques centralizados y descentralizados.

Por último, se exponen los principios generales de la toma de decisiones en carteras y se revisan aplicaciones relacionadas de aprendizaje por refuerzo y aprendizaje por refuerzo multiagente. Esta revisión permite identificar las aportaciones, limitaciones y retos que fundamentan el planteamiento desarrollado en los capítulos posteriores.

2.1 Mercados y plataformas de activos digitales

Los bienes virtuales pueden tener utilidad dentro de un videojuego y, cuando las reglas lo permiten, circular entre personas usuarias a través de mecanismos de intercambio. La investigación sobre consumo de bienes virtuales muestra que la demanda no depende solo de una función dentro del juego. También intervienen la apariencia, la rareza, la identidad social y la percepción de valor del objeto [1, 2].

Por ejemplo, un atuendo para un avatar o una apariencia para un arma modifica su presentación visual sin cambiar las reglas de juego. Un objeto funcional puede dar acceso a una capacidad, una zona o un recurso dentro del juego, proporcionando ventaja al usuario. También existen objetos coleccionables cuyo interés depende de una serie limitada, su rareza o la demanda de la comunidad. Algunos bienes son consumibles, como cajas, sobres o mejoras, y desaparecen o se transforman al utilizarlos.

En los mercados de activos digitales hay dos formas de anunciar un intercambio:

- Una persona usuaria que posee un activo digital y desea venderlo publica una **oferta**. En esa publicación ofrece el activo digital e indica el precio de venta para que otra persona pueda comprarlo.

- Una persona usuaria que desea adquirir un activo digital publica una **orden de compra**. En esa publicación ofrece una cantidad máxima de dinero por un activo digital concreto.

Cuando se completa el intercambio, la plataforma o el servicio de pago puede cobrar costes de transacción, como una comisión por la compra o la venta. Las reglas de cada plataforma determinan cómo se publican, consultan y aceptan estas publicaciones.

Para estudiar una posible compraventa, el precio de compra y el precio de venta deben expresarse en una moneda común. Las comisiones y la conversión de moneda modifican el resultado económico de la operación, por lo que una diferencia entre dos precios publicados no basta para determinar su beneficio neto. El volumen de intercambios indica cuántas unidades de un activo digital se han vendido durante un periodo y aporta información sobre la actividad observada para ese activo.

Algunas plataformas aplican un periodo de bloqueo de intercambio o *trade hold*, durante el cual un activo digital recién adquirido no puede venderse o transferirse. Mientras el bloqueo permanece activo, el importe empleado en la compra constituye capital bloqueado y no puede reutilizarse en otra operación. Estas condiciones afectan al tiempo necesario para completar una compraventa y al saldo disponible de la cartera.

Los mercados de activos digitales pueden clasificarse en centralizados y descentralizados según quién administra la cuenta, los activos digitales asociados a ella y las reglas de intercambio:

- **Mercados centralizados:** una empresa o plataforma administra las cuentas, los activos digitales o las reglas de intercambio. Dentro de esta categoría se encuentran:
 - **Mercados cerrados:** los activos digitales se usan dentro del videojuego, pero no se intercambian entre personas usuarias. Por tanto, no constituyen un **mercado secundario** (un mercado en el que las personas usuarias se revenden activos digitales entre sí) ni existe un precio de reventa observable.
 - **Mercados integrados:** el mercado está incluido dentro de la plataforma que administra la cuenta y los activos digitales asociados a ella. Esa plataforma establece las reglas para publicar ofertas y órdenes de compra.
 - **Plataformas externas:** proporcionan un espacio de intercambio distinto de la plataforma donde se gestionan originalmente la cuenta y los activos digitales. Aplican sus propias reglas, precios y mecanismos de pago.
- **Mercados descentralizados:** la gestión del activo y las reglas de intercambio no dependen de una plataforma central. Habitualmente emplean una infraestructura distribuida para registrar la propiedad y las operaciones.

Los mercados integrados están presentes en distintos ecosistemas de videojuego y distribución. La Tabla 2.1 presenta algunos ejemplos.

Mercado	Ecosistema	Activos habituales
Steam Community Market	Steam	Activos cosméticos, cajas y pegatinas
Roblox Marketplace	Roblox	Prendas digitales, accesorios y animaciones
Auction House	World of Warcraft	Equipo, materiales y moneda
EVE Online Market	EVE Online	Naves, módulos y recursos

Tabla 2.1: Ejemplos de mercados integrados.

Aunque sus activos digitales y reglas son diferentes, estos mercados gestionan los activos digitales asociados a las cuentas y el mecanismo de intercambio dentro del mismo ecosistema [3, 4, 5, 6].

Las plataformas externas ofrecen espacios de intercambio independientes para activos digitales transferibles. La Tabla 2.2 presenta algunos ejemplos.

Plataforma	Especialización	Modalidad
BUFF	Activos digitales transferibles	Compra y venta entre usuarios
Skinport	Activos cosméticos de videojuegos	Mercado con precios publicados
BitSkins	Activos digitales transferibles	Mercado de terceros
DMarket	Activos digitales de videojuegos	Compra y venta entre usuarios

Tabla 2.2: Ejemplos de plataformas externas.

Las dos tablas no deben interpretarse como pares equivalentes, ya que cada una reúne ejemplos independientes de una configuración de intercambio [7, 8, 9, 10].

2.2 Aprendizaje supervisado y no supervisado

El aprendizaje supervisado es un método que aprende a partir de ejemplos en los que se conoce tanto la información de entrada como el resultado que debe estimarse. Cada ejemplo se representa mediante una entrada x y una etiqueta y , que es el resultado conocido asociado a esa entrada. Durante el entrenamiento, el modelo ajusta sus parámetros para aprender la relación entre ambas variables.

Una vez entrenado, el modelo aplica la relación aprendida a nuevas entradas y genera una estimación:

$$\hat{y} = f_{\theta}(x), \quad (2.1)$$

donde x representa la información de entrada, f_{θ} el modelo con parámetros θ y $\hat{y}$ el resultado estimado. Cuando el resultado pertenece a una categoría, el problema se denomina clasificación. Cuando el resultado es un valor numérico, se denomina regresión. En un problema de clasificación binaria, el modelo también puede estimar la probabilidad de que una entrada pertenezca a la clase positiva:

$$\hat{p} = \mathbb{P}(Y = 1 \mid x). \quad (2.2)$$

En la Ecuación 2.2, Y representa el resultado que se desea clasificar, $Y = 1$ representa la clase positiva y $\hat{p}$ es la probabilidad estimada de pertenecer a esa clase. El entrenamiento consiste en seleccionar los parámetros que reducen el error entre las estimaciones y los resultados conocidos del conjunto de ejemplos. Si se dispone de n ejemplos, este ajuste puede expresarse mediante una función de pérdida $\mathcal{L}$:

$$\theta^* = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(y_i, f_{\theta}(x_i)). \quad (2.3)$$

En la Ecuación 2.3, n es el número de ejemplos, x_i es la entrada del ejemplo i , y_i su resultado conocido y $\mathcal{L}$ mide el error entre ese resultado y la estimación del modelo. El valor θ^* representa los parámetros que minimizan el error medio. La ecuación no fija un único modelo ni una única forma de medir el error. La función de pérdida se elige según la tarea. Por ejemplo, puede penalizar la diferencia entre un valor estimado y el valor observado en regresión, o penalizar una clasificación incorrecta cuando las etiquetas representan categorías.

El aprendizaje no supervisado parte de datos de entrada sin una etiqueta o resultado conocido asociado. Su objetivo es descubrir estructuras presentes en los datos, como grupos de observaciones con características parecidas, relaciones entre variables o comportamientos poco habituales. Por ello, puede utilizarse para explorar un conjunto de datos, agrupar elementos similares o detectar observaciones anómalas.

La diferencia esencial entre ambos enfoques se encuentra en la información disponible durante el entrenamiento. El aprendizaje supervisado aprende a relacionar una entrada con un resultado conocido, mientras que el aprendizaje no supervisado describe la estructura de los datos sin partir de un resultado objetivo. Ambos enfoques pueden utilizarse de forma complementaria, por ejemplo, cuando los grupos o patrones identificados mediante aprendizaje no supervisado se incorporan como información adicional a un modelo supervisado. Una estimación o una probabilidad no constituye por sí sola una decisión, ya que no incorpora automáticamente las restricciones ni las consecuencias posteriores de una acción.

2.3 Aprendizaje por refuerzo

El aprendizaje por refuerzo (RL, por sus siglas en inglés de *Reinforcement Learning*) estudia problemas de decisión secuencial en los que un agente aprende mediante su interacción con un entorno [11]. En cada instante t , el agente recibe la información disponible, denominada estado s_t , selecciona una acción a_t y recibe una recompensa r_{t+1} . Como consecuencia de esa acción, el entorno pasa a un nuevo estado s_{t+1} . A diferencia del aprendizaje supervisado, donde un modelo aprende a reproducir etiquetas históricas, en RL el agente aprende qué acciones conviene seleccionar a lo largo de una secuencia de decisiones.

Esta interacción puede representarse mediante un proceso de decisión de Markov, o MDP, definido por el estado, las acciones, la transición, la recompensa y el factor de descuento:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma), \quad 0 \leq \gamma < 1. \quad (2.4)$$

En la Ecuación 2.4, $\mathcal{M}$ representa el proceso de decisión de Markov, $\mathcal{S}$ es el conjunto de estados posibles y $\mathcal{A}$ el conjunto de acciones disponibles. La función de transición $P(s' | s, a)$ expresa la probabilidad de pasar del estado s al estado s' después de ejecutar la acción a . La función $R(s, a, s')$ asigna la recompensa obtenida en esa transición. El factor γ determina cuánto peso reciben las recompensas futuras frente a las inmediatas.

El comportamiento que aprende el agente se denomina política y se representa por π . En una política estocástica, $\pi(a | s)$ expresa la probabilidad de seleccionar la acción a cuando el agente se encuentra en el estado s :

$$\pi(a | s) = \mathbb{P}(A_t = a | S_t = s). \quad (2.5)$$

La Figura 2.1 representa el ciclo básico de interacción. El estado del entorno se entrega al agente, el agente selecciona una acción y el entorno actualiza el estado y proporciona la recompensa correspondiente.

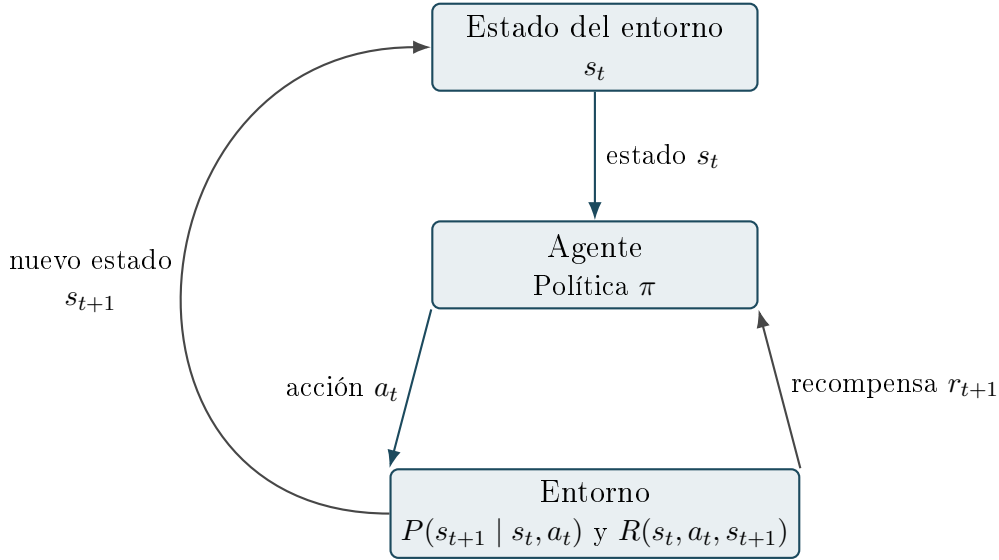


Figura 2.1: Ciclo de interacción entre un agente y un entorno en aprendizaje por refuerzo.

El retorno mide la recompensa acumulada durante un episodio. Si T representa el último instante del episodio, el retorno desde el instante t se define como:

$$G_t^\pi = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}. \quad (2.6)$$

En la Ecuación 2.6, r_{t+k+1} es la recompensa recibida después de la acción tomada en cada instante futuro y γ^k es el descuento aplicado a esa recompensa. Por tanto, una recompensa inmediata tiene más peso que una recompensa situada varios instantes después. El objetivo de RL es aprender la política que maximiza el retorno esperado desde el estado inicial:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} [G_0^\pi]. \quad (2.7)$$

En la Ecuación 2.7, π^* es la política buscada y $\mathbb{E}_\pi$ representa el valor medio del retorno al seguir la política π . La ecuación no busca únicamente la acción con una recompensa inmediata mayor. Busca una política que tenga en cuenta las consecuencias de las decisiones posteriores. La propiedad de Markov no implica que el agente conozca el futuro, sino que el estado reúne la información necesaria para decidir en el instante actual.

Durante el aprendizaje, el agente debe equilibrar exploración y explotación. La exploración consiste en probar acciones cuya utilidad todavía es incierta para conocer sus consecuencias. La explotación consiste en seleccionar las acciones que, según lo aprendido hasta ese momento, tienen un retorno esperado mayor. Explorar en exceso puede acumular decisiones poco útiles, mientras que explotar demasiado pronto puede impedir descubrir alternativas mejores.

2.4 Aprendizaje por refuerzo multiagente

Un sistema multiagente reúne dos o más agentes que actúan sobre un mismo entorno. Estos agentes pueden colaborar, competir o tomar decisiones independientes. En un sistema cooperativo comparten un objetivo global, aunque cada agente disponga de información y responsabilidades distintas [12, 13]. El aprendizaje por refuerzo multiagente, o MARL, amplía la formulación de RL para representar estos sistemas.

Un modelo genérico de MARL puede expresarse como una extensión del proceso de decisión de Markov:

$$\mathcal{M} = (\mathcal{N}, \mathcal{S}, \{\mathcal{A}^i\}_{i \in \mathcal{N}}, P, \{R^i\}_{i \in \mathcal{N}}, \gamma). \quad (2.8)$$

En la Ecuación 2.8, $\mathcal{N}$ es el conjunto de agentes, $\mathcal{S}$ es el conjunto de estados globales y $\mathcal{A}^i$ es el conjunto de acciones disponibles para el agente i . La función P describe la transición del entorno después de la acción conjunta de los agentes. Cada función R^i calcula la recompensa que recibe el agente i y γ es el factor de descuento. En cada instante, los agentes generan una acción conjunta:

$$\mathbf{a}_t = (a_t^1, \dots, a_t^N), \quad r_{t+1}^i = R^i(s_t, \mathbf{a}_t, s_{t+1}). \quad (2.9)$$

En la Ecuación 2.9, N es el número de agentes, $\mathbf{a}_t$ reúne las acciones tomadas en el instante t y r_{t+1}^i es la recompensa que recibe el agente i tras la transición al estado s_{t+1} . Esta formulación permite distinguir cómo se reparten los incentivos entre los agentes.

En un problema plenamente cooperativo todos los agentes reciben la misma recompensa:

$$r_t^1 = r_t^2 = \dots = r_t^N = r_t. \quad (2.10)$$

En la Ecuación 2.10, r_t representa la recompensa común. Esta estructura alinea explícitamente a todos los agentes con el mismo resultado. En un problema cooperativo, en cambio, cada agente puede recibir una recompensa individual distinta. Estas recompensas no tienen por qué coincidir, pero deben diseñarse para que los comportamientos que incentivan contribuyan al objetivo compartido. La distinción es relevante porque

una recompensa común favorece la cooperación directa, mientras que las recompensas individuales permiten reflejar responsabilidades diferentes y pueden dificultar la coordinación si no están bien alineadas.

MARL introduce dificultades que no aparecen con un único agente. Cada agente aprende y modifica su política mientras los demás también modifican las suyas. Por ello, desde el punto de vista de un agente, las consecuencias de una acción pueden cambiar aunque el entorno físico permanezca igual. Este fenómeno se denomina no estacionariedad. Además, un agente puede decidir a partir de una observación local que no contiene toda la información del estado global, lo que dificulta atribuir a cada decisión la parte del resultado que le corresponde.

La información disponible puede repartirse de forma distinta durante el entrenamiento y la ejecución. El entrenamiento es la fase en la que las políticas se ajustan a partir de la experiencia obtenida en el entorno. La ejecución es la fase posterior en la que las políticas ya entrenadas seleccionan acciones. En el entrenamiento centralizado y ejecución centralizada, conocido como CTCE, una entidad central recibe el estado global y selecciona la acción conjunta tanto durante el entrenamiento como durante la ejecución. Esta configuración facilita la coordinación porque concentra toda la información en una única decisión, pero exige que el estado global y la entidad central estén disponibles cuando el sistema actúa. Por ejemplo, un coche autónomo puede utilizar un controlador central que recibe toda la información disponible y decide la aceleración, el frenado y la dirección tanto durante el entrenamiento como durante la conducción.

El paradigma de entrenamiento centralizado y ejecución descentralizada, conocido como CTDE, separa ambas funciones. Durante el entrenamiento, un modelo evaluador puede utilizar el estado global y las acciones de todos los agentes para estimar la calidad esperada de su coordinación. Durante la ejecución, cada agente selecciona su acción a partir de su propia observación local. Por ejemplo, varios coches autónomos pueden entrenarse con un modelo evaluador que conoce la posición, la velocidad y las acciones de todos ellos. Cuando circulan, cada coche decide utilizando únicamente sus propios sensores. Este enfoque permite entrenar con información adicional sin exigir que esa información esté disponible cuando el sistema toma decisiones [14, 15]. El número de coches no determina el paradigma. Varios coches controlados por una única entidad central seguirían un enfoque CTCE. Por tanto, CTCE y CTDE representan dos formas distintas de distribuir la información y la responsabilidad de decisión entre los agentes.

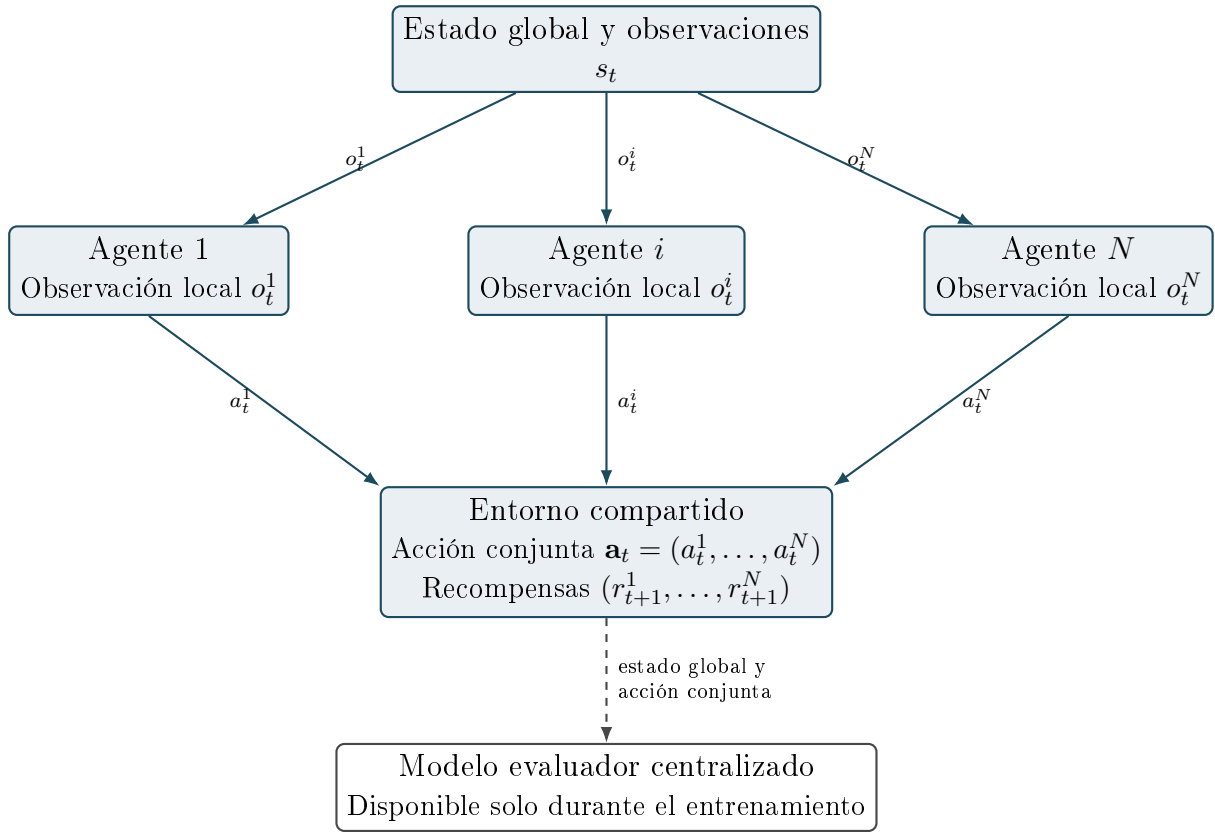


Figura 2.2: Interacción general en MARL con entrenamiento centralizado y ejecución descentralizada.

La Figura 2.2 muestra una interacción general de este tipo. Las líneas continuas representan el intercambio necesario para ejecutar la política de cada agente. Las líneas discontinuas representan la información adicional que utiliza el modelo evaluador únicamente durante el entrenamiento.

2.5 Gestión de carteras y riesgo

La gestión de una cartera estudia cómo las decisiones sobre varios activos y el saldo disponible afectan al conjunto de recursos. No basta con evaluar cada activo de forma aislada. Una decisión de compra o venta de un activo puede parecer atractiva por sí sola y, sin embargo, puede aumentar demasiado la dependencia de la cartera respecto a un activo, una categoría o una plataforma.

La teoría moderna de carteras plantea que la rentabilidad y el riesgo deben analizarse conjuntamente [16]. La rentabilidad expresa el resultado obtenido en relación con los recursos utilizados. El riesgo recoge la incertidumbre y las posibles variaciones desfavorables que pueden afectar a ese resultado. Por ello, la gestión de carteras busca equilibrar ambos elementos en lugar de maximizar únicamente el resultado esperado de una decisión individual.

La asignación de recursos determina qué parte de la cartera se dedica a cada alternativa y qué parte permanece disponible. Una cartera concentrada destina una proporción elevada de sus recursos a pocos activos, categorías o plataformas. Esta concentración

puede aumentar el resultado si los activos seleccionados evolucionan favorablemente, pero también aumenta el efecto de una variación desfavorable sobre el conjunto de la cartera.

La diversificación consiste en repartir los recursos entre alternativas distintas para reducir esa dependencia. No elimina el riesgo, pero evita que el resultado global dependa por completo de una sola alternativa. La exposición permite cuantificar esa dependencia al indicar qué parte de los recursos está vinculada a un activo, una categoría o una plataforma. Ambas nociones permiten comparar distintas distribuciones de una misma cartera.

La composición de la cartera no es necesariamente permanente. El reequilibrio consiste en revisar la distribución de los recursos y ajustarla cuando deja de responder al nivel de riesgo previsto. Esta revisión puede deberse a cambios en el valor de los activos, a la aparición de nuevas alternativas o a cambios en la información disponible. Al evaluar un reequilibrio deben considerarse los costes de transacción, ya que comprar o vender para modificar la cartera reduce el resultado obtenido.

También interviene el horizonte temporal, que determina durante cuánto tiempo se prevé mantener una decisión antes de revisarla. Un horizonte más amplio puede admitir variaciones de precio durante más tiempo, mientras que uno breve requiere que los recursos estén disponibles antes.

Por tanto, la gestión de carteras combina la rentabilidad esperada, la concentración de los recursos, el plazo de las decisiones y los costes asociados a modificarlas.

2.6 Aplicaciones y trabajos relacionados

El aprendizaje automático se ha aplicado a problemas de análisis y decisión en los que los datos cambian con el tiempo. El aprendizaje supervisado permite estimar una variable a partir de observaciones históricas. El aprendizaje no supervisado permite descubrir agrupaciones o patrones en datos sin etiqueta. El aprendizaje por refuerzo permite estudiar decisiones encadenadas cuando una acción modifica las condiciones de las decisiones posteriores.

En la gestión automatizada de carteras, los métodos de aprendizaje por refuerzo se emplean para representar decisiones sucesivas de asignación, mantenimiento o modificación de recursos. La evaluación de estos métodos requiere conservar el orden temporal de los datos y reproducir las condiciones en las que se habría tomado cada decisión. También es necesario incorporar los costes de transacción, pues una estrategia que parece favorable antes de aplicarlos puede dejar de serlo después [17].

El aprendizaje por refuerzo multiagente se ha utilizado para estudiar situaciones en las que varios responsables de decisión comparten un mismo entorno. Estos responsables pueden cooperar, competir o combinar ambos comportamientos. La coordinación permite repartir una tarea compleja entre políticas distintas, aunque añade dificultades como la dependencia entre las decisiones de los agentes y la asignación del resultado colectivo a cada uno de ellos [14, 15].

Estos trabajos muestran que la utilidad de un método no debe valorarse solo por el resultado que obtiene en los datos empleados para entrenarlo. También debe comprobarse su comportamiento con datos no utilizados durante el entrenamiento, su sensibilidad

a cambios en las condiciones del entorno y la claridad de los supuestos de evaluación. Estos criterios permiten comparar métodos de aprendizaje y de decisión de forma reproducible.

Contextualización del caso de estudio: activos digitales en Counter-Strike 2

Este capítulo concreta el marco general presentado en el Capítulo 2 mediante el caso de estudio de *Counter-Strike 2*, un videojuego multijugador desarrollado y distribuido por Valve [18]. Las personas usuarias pueden obtener y utilizar artículos digitales asociados a sus cuentas. Los artículos que se estudian se intercambian en Steam Community Market, un mercado secundario integrado en la plataforma Steam, y en BUFF, una plataforma externa que opera un mercado secundario. Este caso permite estudiar decisiones de compraventa cuando el artículo, el mercado secundario y las reglas de intercambio condicionan conjuntamente el resultado de una operación.

El interés del caso se encuentra en la fragmentación de precios entre mercados secundarios. Un mismo artículo puede publicarse a precios distintos en dos mercados, lo que permite plantear una compra en el mercado más barato y una venta posterior en el mercado con mayor precio. En un mercado bursátil líquido, las diferencias de precio de una misma acción suelen corregirse rápidamente. La existencia de plataformas separadas que administran estos mercados y aplican reglas de intercambio propias hace que *Counter-Strike 2* sea un caso de estudio adecuado para analizar si una diferencia de precios puede convertirse en un beneficio neto.

El caso de estudio se centra en artículos con precios registrados en los mercados secundarios de BUFF y Steam Community Market, cuyas publicaciones pueden verificarse como el mismo artículo. El capítulo describe los artículos, los mercados y las condiciones de intercambio que delimitan este análisis. No presenta todavía la arquitectura, las expresiones económicas ni el simulador, que se desarrollan en los capítulos posteriores.

3.1 Justificación y alcance

El motivo de seleccionar *Counter-Strike 2* es analizar decisiones de compraventa de activos digitales que persiguen generar un beneficio neto a partir de diferencias de precio entre mercados secundarios. Una vez comprobado que dos publicaciones se refieren exactamente al mismo artículo, una diferencia entre sus precios permite plantear una compra en el mercado más barato y una venta posterior en el mercado con mayor precio. El objetivo no es asumir que toda diferencia de precios es rentable, sino determinar cuáles merecen evaluarse como posibles operaciones.

Esta situación no es exclusiva de los activos digitales, pero presenta características distintas de un mercado bursátil líquido. Las unidades de una misma acción ordinaria son intercambiables entre sí y las diferencias de precio para el mismo instrumento suelen corregirse rápidamente. En cambio, los mercados secundarios de activos digitales se

encuentran administrados por plataformas separadas que publican información y aplican reglas de intercambio propias. Esta fragmentación permite observar diferencias de precio más visibles y convierte a *Counter-Strike 2* en un caso adecuado para el objetivo de este trabajo.

El análisis se limita a los artículos con precios registrados en BUFF y Steam Community Market cuyas publicaciones pueden verificarse como el mismo artículo. Esta delimitación permite estudiar una comparación concreta y trazable sin pretender representar todos los artículos de *Counter-Strike 2*, todas las plataformas ni todos los mecanismos de intercambio del videojuego.

3.2 Artículos y atributos de Counter-Strike 2

Los artículos de este caso de estudio pertenecen a las categorías cosméticas y coleccionables descritas en el Capítulo 2. Los artículos cosméticos no modifican las reglas de una partida, pero pueden tener precios diferentes según sus características y el volumen de intercambios existente. Los principales tipos de artículos del caso de estudio son:

- **Skins:** artículos que cambian el diseño visual de un arma. Se pueden equipar durante una partida, pero no alteran ninguna regla del juego.
- **Cuchillos:** artículos que sustituyen el aspecto del cuchillo básico equipado por el personaje. Su categoría y acabado hacen que sean especialmente relevantes como artículos coleccionables.
- **Gaantes:** artículos que modifican el aspecto de las manos del personaje. Pueden combinarse visualmente con un cuchillo o con una *skin*.
- **Pegatinas:** elementos que se aplican a armas compatibles para personalizarlas. Su diseño y la colección de la que proceden pueden influir en su valor de mercado.
- **Cajas:** contenedores que pueden intercambiarse o abrirse. Al abrir una caja se consume y el usuario obtiene un artículo de la colección asociada.

El interés de estos artículos no depende solo de su utilidad dentro del videojuego. También intervienen la disponibilidad, la rareza, la colección a la que pertenecen y la demanda existente en los mercados. Por este motivo, dos artículos del mismo tipo pueden mantener precios muy diferentes. El caso de estudio se centra en los artículos que pueden identificarse y compararse entre los mercados secundarios seleccionados.

Para comparar publicaciones de dos mercados secundarios es necesario determinar si ambas se refieren exactamente al mismo artículo. El nombre base no siempre es suficiente. Por ejemplo, dos artículos denominados «AK-47 | Slate» pueden corresponder a condiciones de desgaste distintas y, por tanto, negociarse con precios distintos.

La identificación del artículo combina el nombre con los atributos que se indican a continuación:

- **Nivel de desgaste:** categoría que describe el desgaste visible del artículo, como *Factory New* o *Battle-Scarred*.
- **Float:** valor numérico asociado al desgaste. En general, un valor más próximo a cero indica un desgaste visual menor.
- **Rareza:** nivel que expresa la dificultad para obtener el artículo. Una rareza mayor no implica necesariamente un precio mayor.
- **StatTrak:** versión especial que incorpora un contador del número de eliminaciones logradas con el arma.

Estos atributos permiten construir un identificador para cada artículo y evitan comparar precios de activos digitales que solo parecen equivalentes. La normalización de este identificador se describe en el Capítulo 5, donde se explica cómo se preparan los datos antes de utilizarlos en los modelos y en la simulación.

3.3 Mercados secundarios y condiciones de intercambio

El caso de estudio combina dos mercados secundarios administrados por plataformas distintas con una fuente auxiliar de descubrimiento. Cada fuente aporta una parte diferente de la información necesaria para estudiar una posible operación:

- **SteamDT** se utiliza para localizar artículos candidatos que posteriormente se verifican en BUFF y Steam Community Market. No proporciona el precio de compra ni el precio de venta de la operación simulada, ni establece sus condiciones económicas [19].
- **Steam Community Market** es el mercado secundario integrado en la plataforma Steam. Publica ofertas de venta y órdenes de compra de artículos asociados a las cuentas de las personas usuarias. En la operación de compraventa, el precio de una oferta de venta observada en este mercado se utiliza como referencia del precio de venta simulado. También aporta información histórica y de actividad de intercambio [3].
- **BUFF** es una plataforma externa que opera un mercado secundario de artículos transferibles. En la operación de compraventa, el precio de una oferta de venta observada en BUFF se utiliza como referencia del precio de compra simulado. Los precios se conservan en CNY y con su conversión a EUR para permitir la comparación, junto con la actividad de intercambio y las órdenes de compra [7].

La operación de compraventa compara una oferta de venta en BUFF con una oferta de venta en Steam Community Market para el mismo artículo. Las órdenes de compra

describen una intención distinta, ya que una persona ofrece dinero hasta un importe máximo para adquirir un artículo. Por ello, se emplean como información complementaria y no como el precio principal de la operación simulada.

Para determinar si esa diferencia de precios puede estudiarse como una operación, ambos precios deben expresarse en una moneda común y el precio de venta debe ajustarse con las comisiones aplicables. Una diferencia entre precios antes de esos ajustes no permite determinar por sí sola el resultado de una operación.

En la comparación entre BUFF y Steam Community Market, el volumen de intercambios registrado en cada mercado secundario permite valorar si la venta prevista puede completarse en el plazo considerado. El simulador también representa el periodo de bloqueo de intercambio cuando se aplica. En ese caso, la venta se pospone y el capital empleado en la compra permanece comprometido hasta que el artículo pueda venderse. Estas condiciones afectan a la selección de candidatos para la simulación.

La comparación de precios se utiliza para seleccionar candidatos que pueden estudiarse en la simulación. No supone que el artículo pueda comprarse o venderse al precio publicado, ni garantiza que ese precio se mantenga hasta completar el intercambio. El Capítulo 5 detalla la captura y normalización de los datos, mientras que los Capítulos 4 y 5 presentan las expresiones económicas y las reglas del simulador.

Modelo de decisión y arquitectura multiagente propuesta

Este capítulo concreta, para el caso de estudio, el modelo de decisión que se utilizará en la propuesta multiagente. En el Capítulo 2 se ha presentado el marco teórico general del aprendizaje supervisado, el aprendizaje por refuerzo y el aprendizaje por refuerzo multiagente [11, 12, 13]. En el Capítulo 3 se explica el caso de *Counter-Strike 2*: qué artículos se estudian, qué plataformas intervienen y qué condiciones afectan a una operación de compraventa [3, 7, 18]. A partir de ese material previo, este capítulo explica cómo se aplican esos conceptos al problema concreto de compraventa simulada.

La estructura del capítulo se separa en dos partes. Primero se define el modelo de decisión aplicado: agentes, estado, observaciones, acciones, transición, recompensa, restricciones y criterio temporal. Después se presenta la arquitectura de la solución, es decir, la organización técnica que permite capturar datos, almacenarlos, producir una **señal supervisada** (valor producido por un modelo entrenado) y ejecutar episodios de decisión simulada.

La evaluación se realiza exclusivamente dentro del simulador. Las acciones modifican una cartera simulada, no una cuenta real ni artículos reales. Por tanto, el capítulo describe una propuesta de decisión y aprendizaje, no un mecanismo de ejecución automática sobre Steam, BUFF ni ninguna otra plataforma [3, 7].

4.1 Problema de decisión del caso de estudio

El problema que aborda este trabajo consiste en decidir, a lo largo del tiempo, cómo actuar ante **oportunidades** de compraventa, entendidas como posibles operaciones que merecen revisarse porque sus datos iniciales indican que podrían generar un beneficio neto. Ante cada oportunidad, el sistema puede comprar, mantener una posición, vender una posición abierta o descartar la operación. Esta definición no equivale a comprar cualquier artículo con una diferencia favorable entre plataformas. Una diferencia de precios solo es el punto de partida: antes de considerarla una operación válida hay que comprobar que ambos precios corresponden al mismo artículo, que los costes permiten conservar margen y que la cartera puede asumir la compra [17].

Decisión secuencial. La pregunta principal del modelo no es si existe una diferencia de precios en un instante aislado, sino qué decisión debe tomarse con los datos y el capital disponible en ese momento. La decisión actual condiciona las decisiones posteriores: una compra reduce el saldo disponible, crea una posición abierta y puede impedir que ese mismo capital se use en otra oportunidad. Por tanto, el problema no se resuelve mirando cada candidato por separado, sino evaluando cómo cada acción modifica la situación de la cartera durante una secuencia temporal.

Uso de cartera y capital bloqueado. Mientras una posición permanece abierta, el capital invertido no puede emplearse en otra alternativa. Si además existe un periodo de bloqueo de intercambio, la venta no puede ejecutarse inmediatamente aunque aparezca un precio favorable. Por este motivo, una decisión aparentemente buena puede empeorar el comportamiento global de la cartera si impide aprovechar operaciones posteriores o aumenta demasiado la exposición a un mismo artículo [16, 17]. Por ejemplo, si se compra una unidad de la *skin* «AK-47 / Slate» por 10 EUR, esos 10 EUR quedan comprometidos en la cartera. Si después aparece una oportunidad de compraventa de la *skin* «M4A1-S / Printstream» por 15 EUR, no se puede utilizar el mismo capital ya invertido en la primera *skin*. Si la cartera no conserva suficiente saldo disponible, la segunda operación deberá descartarse o aplazarse hasta que exista liquidez.

Compra, mantenimiento, venta y descarte. El carácter secuencial también afecta a las ventas. Una operación de compraventa no se evalúa por completo en el momento de comprar, ya que el resultado económico se conoce cuando la posición se cierra. Hasta entonces, la cartera conserva una posición abierta, soporta el capital bloqueado y depende de que exista una salida posterior. Mantener o descartar también son decisiones relevantes: evitar una compra puede proteger la cartera y conservar el dinero disponible, pero también puede hacer que se pierda una operación que habría producido una rentabilidad positiva.

Simulación, no ejecución real. El alcance del problema se limita a decisiones simuladas. El sistema no envía órdenes a los mercados ni garantiza que un precio publicado pueda ejecutarse en la práctica. Su objetivo es construir un entorno reproducible en el que se puedan comparar decisiones con las mismas reglas económicas, las mismas restricciones de cartera y el mismo orden temporal de los datos. Así se evita confundir una recomendación experimental con una actuación directa sobre una cuenta real [20].

Papel de la señal supervisada. El modelo de decisión utiliza una señal supervisada como información auxiliar: una probabilidad calculada a partir de datos históricos para estimar si un candidato puede ser interesante. Esta señal es auxiliar porque no decide por sí sola. Una probabilidad alta no equivale a una orden de compra, sino a un dato adicional que los agentes pueden usar al evaluar el candidato. La decisión final depende también de los costes de compra y venta, el saldo disponible, las posiciones abiertas, el capital bloqueado y las restricciones de riesgo. Por ejemplo, un candidato puede tener una señal supervisada favorable y aun así ser rechazado si no hay saldo suficiente, si la cartera ya concentra demasiado capital en ese artículo o si el activo permanecería bloqueado durante un periodo que aumenta el riesgo operativo. Por tanto, el aprendizaje supervisado se usa como una entrada más del estado observado por los agentes, no como sustituto del proceso de decisión multiagente.

Con esta descripción, el problema queda preparado para expresarse como un modelo MARL aplicado. La Sección 4.2 concreta los agentes que intervienen, la información que constituye el estado, la parte que observa cada agente, qué acciones se permiten y cómo el simulador transforma cada decisión en una nueva situación de cartera.

4.2 Modelo MARL aplicado al caso de estudio

Partiendo de la formulación general presentada en la Sección 2.4, el problema de este trabajo se modela como un sistema MARL con observabilidad parcial. En un proceso de decisión de Markov parcialmente observable, conocido como POMDP (*Partial Observability Markov Decision Process*), el entorno mantiene un estado real, pero el agente solo recibe una observación incompleta de ese estado [21]. En este trabajo se utiliza esa idea en un contexto multiagente: existe un estado global de mercado y cartera, pero cada agente recibe solo la parte de información que necesita para cumplir su responsabilidad [12, 13].

En cada instante del episodio, el entorno presenta un candidato de compraventa construido a partir de datos de mercado. Ese candidato se combina con la situación actual de la cartera simulada y, cuando está disponible, con una señal supervisada calculada a partir de datos históricos. Los agentes no modifican directamente los datos de mercado: proponen acciones sobre la cartera simulada. Después, el simulador comprueba si la acción conjunta es válida, calcula el estado siguiente y devuelve las recompensas correspondientes.

El modelo aplicado se expresa mediante la siguiente tupla:

$$\mathcal{M}_{\text{CS2}} = (\mathcal{N}, \mathcal{S}, \{\mathcal{O}^i\}_{i \in \mathcal{N}}, \{\mathcal{A}^i\}_{i \in \mathcal{N}}, P_{\text{sim}}, \{R^i\}_{i \in \mathcal{N}}, \gamma). \quad (4.1)$$

- **Elementos de la tupla.** En la Ecuación 4.1, $\mathcal{N}$ es el conjunto de agentes, $\mathcal{S}$ es el conjunto de estados globales, $\mathcal{O}^i$ es el conjunto de observaciones locales que recibe el agente i , $\mathcal{A}^i$ es su conjunto de acciones, P_{sim} es la función de transición implementada por el simulador, R^i es la recompensa recibida por el agente i y γ es el factor de descuento. La presencia de $\mathcal{O}^i$ indica que el modelo no asume observación completa por parte de todos los agentes: el entorno conserva el estado global, pero cada agente decide con una vista parcial.
- **Agentes.** El conjunto de agentes se define como $\mathcal{N} = \{\text{Scout}, \text{Trader}, \text{Portfolio}\}$. Scout revisa si el candidato merece atención; Trader decide la acción operativa sobre la cartera; y Portfolio valida si la propuesta respeta saldo, exposición, posiciones abiertas y restricciones de riesgo. La separación no significa que trabajen en procesos independientes. Los tres agentes actúan sobre el mismo entorno y sus decisiones se interpretan juntas en cada instante.
- **Estado global.** El estado global $s_t \in \mathcal{S}$ reúne toda la información que el entorno necesita para simular correctamente el instante t . En este trabajo se compone de tres bloques:
 - **Información dinámica de mercado:** plataforma de compra, plataforma de venta, tipo de precio, precio de compra normalizado, valor neto de salida estimado, retorno actual, destino del saldo, cantidad disponible y señales de actividad. Esta información no es una configuración fija. Representa lo que el sistema conoce de un artículo en una fecha concreta del episodio. Si el mismo artículo aparece de nuevo en otra fecha, el estado puede cambiar porque cambian sus precios, su disponibilidad o el volumen observado.

- **Información dinámica de cartera:** efectivo disponible, efectivo previsto tras la compra, capital bloqueado, exposición del candidato, posiciones desbloqueadas vendibles y posiciones todavía bloqueadas. Esta parte cambia por las decisiones simuladas anteriores: una compra reduce saldo y abre una posición; una venta cierra una posición y recupera efectivo neto; y el paso del tiempo puede desbloquear activos que antes no podían venderse.
- **Señal supervisada:** probabilidad estimada por el modelo supervisado y un indicador que señala si esa probabilidad está disponible en el episodio. La señal forma parte del estado porque es información que los agentes pueden observar, pero su papel sigue siendo auxiliar: informa sobre el candidato, no ejecuta una operación ni sustituye a la decisión multiagente.

Además de estos bloques, cada episodio utiliza parámetros configurados antes de empezar, como comisiones, horizonte temporal, duración del bloqueo de intercambio, límites de riesgo y saldo inicial. Estos parámetros no son decisiones de los agentes ni cambian en cada paso, pero condicionan cómo el simulador calcula transiciones y recompensas.

- **Observaciones locales.** Cada agente recibe una observación local $o_t^i \in \mathcal{O}^i$, es decir, una selección del estado global adaptada a su responsabilidad. Scout observa principalmente información del candidato, actividad de mercado y señal supervisada. Trader observa esas señales junto con información de salida, saldo disponible y posiciones del mismo artículo. Portfolio observa ratios de cartera, avisos de riesgo, número de incumplimientos y posiciones vendibles. Esta distinción evita asumir que todos los agentes conocen toda la información del sistema.
- **Acciones.** El conjunto de acciones se define por rol y contiene siete acciones disponibles repartidas entre los tres agentes:
 - **Scout:** *ignorar candidato o marcar candidato.*
 - **Trader:** *mantener cartera, comprar una unidad o vender una unidad.*
 - **Portfolio:** *rechazar propuesta o aprobar propuesta.*

En cada instante, cada agente elige una de sus acciones disponibles. La combinación de las tres decisiones forma la **acción conjunta** $\mathbf{a}_t = (a_t^S, a_t^T, a_t^P)$. Como hay dos acciones de Scout, tres de Trader y dos de Portfolio, existen hasta $2 \cdot 3 \cdot 2 = 12$ combinaciones posibles. No todas producen una operación económica. Por ejemplo, una compra simulada solo se intenta cuando Scout marca el candidato, Trader propone comprar una unidad y Portfolio aprueba la propuesta. Una venta simulada solo se intenta cuando Trader propone vender una unidad, Portfolio aprueba la propuesta y existe una posición desbloqueada del mismo artículo. En el resto de combinaciones, la cartera se mantiene o se registra una decisión no ejecutada.

- **Simulador y transición.** Las acciones no las devuelve el simulador. Las acciones las generan las políticas de los agentes a partir de sus observaciones locales. El simulador recibe la acción conjunta, comprueba qué caso representa y calcula la transición $P_{\text{sim}}(s_{t+1} \mid s_t, \mathbf{a}_t)$. La transición es la regla que determina el estado resultante s_{t+1} después de aplicar la decisión sobre el estado actual s_t .

- **Compra válida:** si la acción conjunta solicita una compra y la propuesta cumple las restricciones, el estado siguiente reduce el efectivo disponible, abre una nueva posición, registra el coste invertido, actualiza la exposición de cartera y guarda la fecha a partir de la cual la unidad podrá venderse.
- **Venta válida:** si la acción conjunta solicita una venta, Portfolio la aprueba y existe una posición desbloqueada del mismo artículo, el estado siguiente cierra esa posición, aplica comisiones, incorpora el importe neto al efectivo disponible y actualiza el beneficio de la operación.
- **Decisión no ejecutada o inválida:** si la acción conjunta no solicita una operación completa, si no hay saldo suficiente, si no existe posición vendible o si se incumple una restricción de riesgo, el estado siguiente conserva la cartera económica sin abrir ni cerrar posiciones. Aun así, el simulador registra el motivo de no ejecución para que la recompensa pueda distinguir entre esperar voluntariamente y proponer una operación imposible.

Esta transición modela el efecto de la decisión sobre la cartera simulada. Los precios históricos y los datos de mercado observados no se modifican por la acción de los agentes; lo que cambia es la cartera, el registro del episodio y, en el siguiente instante, el candidato de mercado que presenta la secuencia temporal.

- **Restricciones de riesgo.** Las restricciones no son un agente adicional ni un elemento nuevo de la tupla. Dentro del modelo aparecen integradas en tres lugares. Primero, forman parte de P_{sim} , porque condicionan si una acción conjunta puede transformarse en una compra válida o debe dejar la cartera sin cambios. Segundo, forman parte de las observaciones de Portfolio, porque este agente recibe avisos de saldo, exposición, bloqueo y posiciones vendibles antes de aprobar o rechazar. Tercero, forman parte de R^i , porque una propuesta inválida puede generar penalizaciones. Antes de ejecutar una compra, el entorno calcula cuatro controles: límite de tamaño de posición, exposición por artículo, exposición por plataforma y reserva mínima de efectivo. Estos controles impiden que una señal supervisada favorable apruebe por sí sola una operación que dejaría la cartera mal distribuida, sin liquidez suficiente o concentrada en un único activo.
- **Recompensa y episodio.** La recompensa de cada agente se define mediante una función $R^i(s_t, \mathbf{a}_t, s_{t+1})$. En este trabajo se usa una recompensa híbrida: una parte común asociada al resultado económico de la cartera y una parte individual asociada a la responsabilidad de cada rol. La parte común se obtiene al cerrar operaciones o al proponer compras inválidas. La parte individual penaliza errores concretos, como marcar candidatos que terminan con pérdidas, no comprar oportunidades viables, vender tarde o aprobar compras que incumplen límites. La Sección 4.5 desarrolla esta función con sus ecuaciones.

El factor de descuento γ mantiene el significado presentado en la Sección 2.3. En este problema es relevante porque la recompensa económica suele aparecer después de la compra, cuando una venta posterior cierra la posición. Un valor de γ alto permite que el entrenamiento relacione decisiones de compra con recompensas futuras, mientras que un valor bajo daría demasiado peso a señales inmediatas [11]. La implementación

experimental utiliza $\gamma = 0,99$, como se detalla en los parámetros de entrenamiento del Anexo A.

Cada **episodio** es una ejecución completa del simulador sobre una secuencia temporal de candidatos. Comienza con una cartera inicial, recorre instantes ordenados por fecha y termina cuando se han procesado todos los datos del periodo seleccionado o cuando se alcanza el límite de pasos configurado. Durante el episodio se actualizan saldo, posiciones abiertas, capital bloqueado, fechas de desbloqueo y métricas de exposición. El objetivo de aprendizaje sigue el criterio general de la Ecuación 2.7: obtener políticas que maximicen el retorno esperado a lo largo del episodio.

4.3 Arquitectura del sistema

Una vez definido el problema MARL, la arquitectura describe cómo se organizan los componentes necesarios para construir el estado de decisión, ejecutar las acciones y registrar sus consecuencias. La arquitectura se organiza en cuatro niveles: captura y persistencia de datos, preparación del estado, decisión de los agentes y aplicación de la decisión simulada. Cada nivel recibe una salida definida del anterior y entrega una entrada definida al siguiente. Esta separación reduce el acoplamiento y permite comprobar cada parte por separado, una práctica habitual en entornos de evaluación reproducible [17, 20].

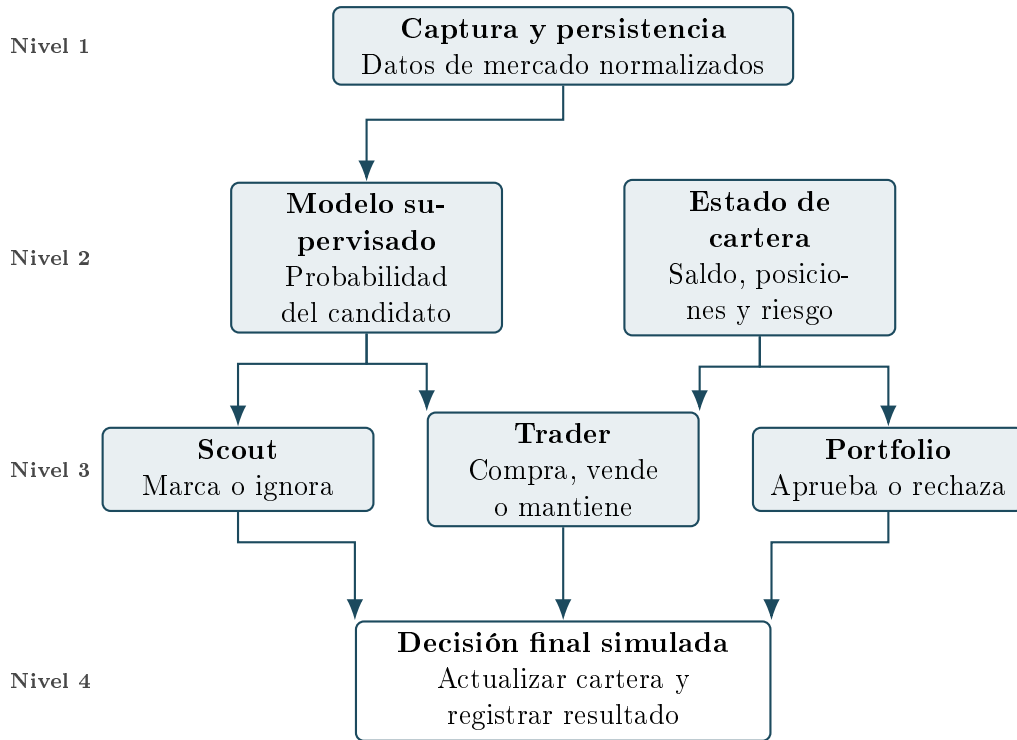


Figura 4.1: Niveles principales de la arquitectura propuesta.

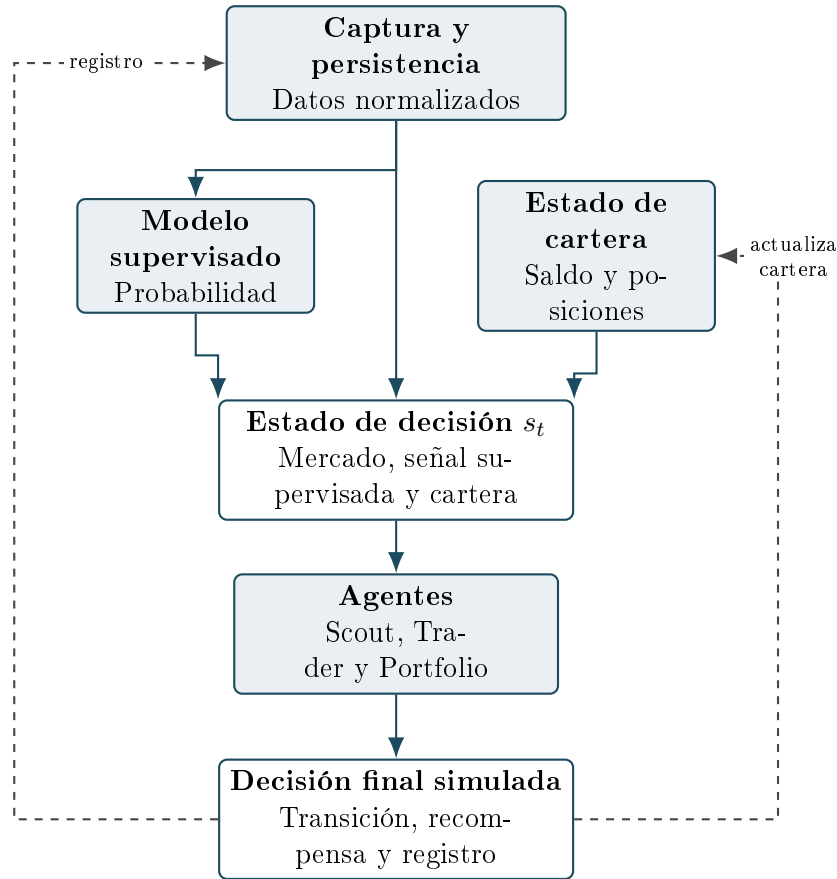


Figura 4.2: Construcción del estado y retroalimentación de la decisión simulada.

La Figura 4.1 resume el recorrido principal por niveles. El primer nivel captura y conserva observaciones de Steam Community Market, BUFF, SteamDT, OCR e importaciones manuales. El segundo nivel prepara la información que alimenta la decisión: el modelo supervisado genera una probabilidad a partir de los datos de mercado y el simulador mantiene el estado de la cartera. El tercer nivel reparte esa información entre Scout, Trader y Portfolio. El último nivel aplica la decisión final simulada, actualiza la cartera y registra el resultado para conservar la trazabilidad del episodio.

La Figura 4.2 separa el detalle que no conviene cargar en la figura general. El estado de decisión s_t combina datos de mercado normalizados, señal supervisada y estado de cartera. El estado de cartera no procede directamente de la captura, sino de la cartera inicial y de las decisiones simuladas anteriores. Por eso la retroalimentación se representa desde la decisión final hacia la cartera, mientras que el registro de resultados vuelve al nivel de persistencia para conservar la historia completa del episodio.

El flujo comienza cuando los componentes de adquisición recopilan datos de los mercados secundarios y de las fuentes auxiliares. Antes de utilizar los datos extraídos, se normalizan y se conservan junto con su instante de captura para construir un histórico trazable. Sobre ese histórico se calculan las variables necesarias para obtener una señal supervisada. Una probabilidad alta indica que una posible operación merece atención, pero los agentes todavía deben comprobar el estado de la cartera y las restricciones antes de aprobar una compra simulada.

La salida del sistema debe ser auditable. Cada predicción, acción y decisión simulada se relaciona con su fuente de datos, la versión del modelo, el episodio y la configuración de simulación. Esta trazabilidad permite distinguir un error de datos, un error del modelo y una decisión inadecuada de la política. La decisión final siempre queda registrada como simulada. La arquitectura puede generar recomendaciones, registros de episodios y métricas, pero no envía órdenes a Steam, BUFF ni ninguna otra plataforma.

Algoritmo 1 Flujo de información y decisión simulada

Entrada: Secuencia temporal de candidatos, estado inicial de cartera y señal supervisada opcional

Salida: Cartera actualizada, recompensa y registro trazable

```

1: for cada candidato  $c_t$  del episodio do
2:   Consultar el candidato ya capturado y normalizado.
3:   Consultar el estado de cartera  $s_t$  y calcular las restricciones aplicables.
4:   Incorporar la señal supervisada cuando esté disponible.
5:   Generar las observaciones locales  $o_t^S$ ,  $o_t^T$  y  $o_t^P$ .
6:   Obtener  $a_t^S$ ,  $a_t^T$  y  $a_t^P$  y formar la acción conjunta  $\mathbf{a}_t$ .
7:   Validar la acción conjunta con las restricciones de riesgo.
8:   Simular la transición hacia  $s_{t+1}$ .
9:   Actualizar saldo, posiciones, capital bloqueado y métricas.
10:  Calcular recompensas y registrar la trazabilidad del paso.
11: end for

```

Cada paso registra la información utilizada, las acciones propuestas y el resultado de la simulación. Este registro permite reconstruir cómo se obtuvo una recomendación y diferenciar la información de partida de las consecuencias que calcula el simulador.

4.4 Agentes de decisión

La decisión simulada se distribuye entre tres agentes MARL con responsabilidades diferenciadas. Cada agente recibe una observación local adaptada a su función y propone una acción dentro del entorno de simulación:

- **Scout:** revisa la plataforma, el tipo de publicación, el precio de compra, el retorno previsto, la disponibilidad y la probabilidad supervisada. Su función es identificar los artículos que merecen atención.
- **Trader:** utiliza la señal de Scout, la información de compra y venta prevista y las posiciones del mismo artículo que ya existen en la cartera para proponer la acción operativa principal.
- **Portfolio:** comprueba el saldo, el capital bloqueado, la exposición prevista, las posiciones que pueden venderse y las restricciones de riesgo antes de validar una propuesta.

Las acciones disponibles para cada agente son las siguientes:

- **Scout:** *ignorar* o *marcar* un artículo.

- **Trader:** *mantener* la cartera, *comprar* una unidad o *vender* una posición desbloqueada del artículo.
- **Portfolio:** *rechazar* o *aprobar* la propuesta.

Algoritmo 2 Decisión cooperativa

Entrada: Artículo candidato c_t y observaciones locales

Salida: Transición de cartera y recompensas híbridas r_t^S , r_t^T y r_t^P

```

1:  $a_t^S \leftarrow \pi_S(o_t^S)$ 
2:  $a_t^T \leftarrow \pi_T(o_t^T)$ 
3:  $a_t^P \leftarrow \pi_P(o_t^P)$ 
4: if  $a_t^S = \text{marcar}$  y  $a_t^T = \text{comprar}$  y  $a_t^P = \text{aprobar}$  y la compra cumple los límites de riesgo
   then
5:   Abrir una posición simulada y bloquear su capital
6: else if  $a_t^T = \text{vender}$  y  $a_t^P = \text{aprobar}$  y existe una posición desbloqueada de  $c_t$  then
7:   Cerrar una posición simulada y abonar su importe neto
8: else
9:   Mantener el saldo y registrar el motivo de no ejecución
10: end if
11: Actualizar el saldo, las posiciones, la exposición y las métricas de riesgo.
12: Calcular la recompensa común y las señales individuales de los tres agentes.
13: Devolver  $r_t^S$ ,  $r_t^T$  y  $r_t^P$  mediante la combinación híbrida.

```

En el Algoritmo 2, c_t representa el artículo considerado en el instante t . Las funciones π_S , π_T y π_P son las políticas de Scout, Trader y Portfolio. Cada política recibe su observación local, representada por o_t^S , o_t^T u o_t^P , y produce una acción, representada por a_t^S , a_t^T o a_t^P .

Una compra simulada requiere que Scout marque el artículo, Trader solicite la compra y Portfolio la apruebe. Una venta simulada requiere la solicitud de Trader, la aprobación de Portfolio y una posición desbloqueada del mismo artículo. Scout no emite una marca nueva para vender porque la posición ya forma parte de la cartera. La recompensa de cada agente incorpora un componente común y otro individual, como se explica en la Sección 4.5.

4.5 Función de recompensa

BUFFF, HAY QUE ESTRUCTURAR ESTA SECCIÓN EN SUBSECCIONES ... NO PUEDES HACER UNA SECCIÓN QUE SEAN 4 PÁGINAS!!

El objetivo del sistema es maximizar el valor de la cartera simulada. La recompensa debe reflejar este objetivo sin incentivar decisiones que incumplan los límites de riesgo, ignoren los costes o no respeten las restricciones de intercambio. Una recompensa basada solo en comprar a un precio bajo o en detectar una subida de precio sería incompleta, porque no tendría en cuenta si puede realizarse una venta posterior, si el capital queda bloqueado o si la compra supera los límites de la cartera [11, 16, 17].

Recompensa cooperativa. La recompensa común mide el resultado económico de una operación de compraventa cuando se completa su ciclo de compra y venta. La compra abre una posición y la venta posterior la cierra. Solo entonces se conoce el importe

recuperado y se puede valorar la operación. Una compra permitida no recibe recompensa cooperativa en ese instante. La única excepción es una propuesta que incumple restricciones, que recibe una penalización inmediata para que los agentes aprendan a no proponer operaciones imposibles.

En las ecuaciones siguientes, t representa el instante actual del episodio y j una operación de compraventa cerrada en ese instante. Los valores a , b y c son coeficientes configurables. Su selección se realizará mediante validación, utilizando episodios distintos de los empleados para entrenar el modelo.

El retorno sobre la inversión, denominado *Return on Investment* (ROI), expresa el beneficio neto de una operación con relación al importe invertido:

$$\text{ROI}_j = \frac{B_j^{\text{neto}}}{I_j}. \quad (4.2)$$

En la Ecuación 4.2, B_j^{neto} es el beneficio neto de la operación j e I_j es el importe invertido para adquirir el artículo, incluidos los costes de compra aplicables. Por ejemplo, si se invierten 100 EUR y la venta posterior devuelve 110 EUR después de aplicar sus costes, el beneficio neto es 10 EUR y $\text{ROI}_j = \frac{10}{100} = 0,10$. El retorno sobre la inversión permite comparar operaciones de importes distintos sin dar más importancia a una operación solo por requerir más capital.

Antes del entrenamiento, el ROI se limita al intervalo entre -1 y 1 . Esto evita que un resultado excepcional determine por sí solo el aprendizaje. En la ecuación siguiente, R_j representa el ROI limitado. Por ejemplo, un ROI de $0,10$ mantiene ese valor, mientras que un ROI de $1,40$ pasa a valer 1 .

La recompensa cooperativa se calcula de la siguiente manera:

$$r_t^{\text{cartera}} = \begin{cases} a \cdot R_j - b \cdot e_j, & \text{si una venta cierra la operación } j, \\ -c \cdot l_t, & \text{si una compra propuesta incumple restricciones,} \\ 0, & \text{en cualquier otro caso.} \end{cases} \quad (4.3)$$

En el primer caso, a determina la importancia del ROI realmente obtenido. El valor e_j es el número de días que la operación j ha permanecido abierta por encima de ocho días de bloqueo obligatorios. Durante los primeros ocho días no se penaliza el tiempo de espera, porque ese periodo corresponde al bloqueo de intercambio habitual. Por ejemplo, si una operación se cierra doce días después de la compra, $e_j = 4$. El coeficiente b indica cuánto se reduce la recompensa por cada día adicional.

En el segundo caso, l_t representa la proporción de límites incumplidos respecto de los límites comprobados. También toma valores entre 0 y 1 . El coeficiente c debe ser suficientemente alto para que proponer una compra inválida no resulte preferible a no comprar.

La proporción de capital que Trader propone destinar a compras se controla mediante un objetivo flexible. La fórmula divide ese capital propuesto entre el valor de la cartera antes de tomar la decisión. Si C_t es el importe total de las compras propuestas por Trader en un instante y V_t es el valor de la cartera antes de esas compras, la proporción propuesta se define como:

$$p_t = \frac{C_t}{V_t}. \quad (4.4)$$

Por ejemplo, si la cartera vale 1.000 EUR antes de comprar y Trader propone comprar por un total de 400 EUR, entonces $C_t = 400$, $V_t = 1,000$ y $p_t = \frac{400}{1,000} = 0,40$. Es decir, Trader propone destinar el 40 % de la cartera a esas compras. El subíndice t solo indica el instante en el que se realiza este cálculo.

El valor u representa la proporción objetivo y d el margen de tolerancia, ambos configurables. No se considera que exista un incumplimiento de este límite mientras se cumpla la siguiente condición:

$$u - d \leq p_t \leq u + d. \quad (4.5)$$

Por tanto, la parte superior de la banda, $u + d$, es el máximo efectivo para este límite flexible. Si Trader propone una proporción superior, esa propuesta se incorpora en l_t como un incumplimiento y Portfolio debe rechazarla. Si Portfolio la aprobara, también recibiría su penalización individual. Los límites de riesgo estrictos se mantienen sin tolerancia. Además, los valores elegidos para u y d deben ser compatibles con dichos límites.

Por ejemplo, una compra de 100 EUR recibe una recompensa cooperativa de 0 en el momento de comprar. Si la venta posterior devuelve 110 EUR después de aplicar sus costes, el beneficio neto es 10 EUR y el ROI es 0,10. Si se vende en el octavo día, $e_j = 0$. Con $a = 0,60$, la recompensa al cerrar la operación es $r_t^{\text{cartera}} = 0,60 \cdot 0,10 = 0,06$. Si la misma venta se realiza cuatro días más tarde, $e_j = 4$, y $b = 0,01$, la recompensa pasa a ser $r_t^{\text{cartera}} = 0,60 \cdot 0,10 - 0,01 \cdot 4 = 0,02$. En cambio, si se propone una compra que incumple una cuarta parte de los límites comprobados, $l_t = 0,25$, y $c = 0,80$, la recompensa es $r_t^{\text{cartera}} = -0,80 \cdot 0,25 = -0,20$.

Recompensa individual. Además de la recompensa común, el entorno calcula una señal individual para cada rol. Estas señales no representan dinero. Usan el ROI, los días adicionales y la proporción de límites incumplidos, por lo que son comparables con la recompensa cooperativa.

La señal individual de Scout evalúa la selección inicial de cada artículo. Antes de mostrar la expresión, se aplican dos reglas sencillas:

- Si Scout marca un artículo y la operación de compraventa posterior termina con un ROI negativo, Scout recibe una penalización igual a ese ROI negativo.
- Si Scout no marca un artículo que era posible comprar con el saldo disponible y que posteriormente habría obtenido un ROI positivo, Scout recibe una penalización igual al ROI que ha dejado pasar.

La segunda regla se aplica de forma independiente a cada artículo. Por ello, Scout no se penaliza por no marcar un artículo rentable cuando el saldo ya se ha empleado en otras compras aprobadas. Puede marcar y el sistema puede comprar varios artículos de bajo precio si hay saldo suficiente. Del mismo modo, puede marcar un único artículo de precio elevado si su compra es viable. En ambos casos, el resultado económico de las operaciones cerradas se incorpora en la recompensa cooperativa.

La expresión de la señal individual de Scout es la siguiente:

$$r_{S,t} = \begin{cases} R_j, & \text{si Scout marcó el artículo de } j \text{ y } R_j < 0, \\ -R_j, & \text{si Scout no marcó el artículo de } j, R_j > 0 \text{ y había saldo suficiente,} \\ 0, & \text{en cualquier otro caso.} \end{cases} \quad (4.6)$$

El primer caso se evalúa cuando se cierra la operación j . Si Scout marcó un artículo que termina con pérdidas, R_j es negativo y su señal individual también lo es. Si la operación es rentable, su señal individual es 0, porque la recompensa cooperativa ya reconoce ese resultado. Pero si es negativo, se aplica un castigo adicional a Scout por haber marcado un artículo que no era rentable.

El segundo caso penaliza una oportunidad perdida. El ROI R_j se calcula con los datos históricos del episodio como si el artículo se hubiera comprado en ese momento. Esta comprobación solo se utiliza durante el entrenamiento. Por ejemplo, si Scout no marca un artículo para el que quedaban 100 EUR de saldo y dicho artículo habría obtenido un ROI de 0,15, recibe $-0,15$. Si ya no quedaba saldo para comprarlo, no recibe esa penalización.

La señal individual de Trader evalúa la ejecución de la compra y de la venta. Para valorar si ha dejado una parte excesiva del capital sin utilizar, se calcula primero el déficit respecto al extremo inferior de la banda:

$$h_t = \begin{cases} \frac{(u - d) - p_t}{u - d}, & \text{si } p_t < u - d, \\ 0, & \text{en cualquier otro caso.} \end{cases} \quad (4.7)$$

El valor h_t toma valores entre 0 y 1. Solo se utiliza si, tras procesar los artículos disponibles, todavía existen artículos marcados, rentables y viables que habrían permitido alcanzar el extremo inferior de la banda. De este modo, no se castiga a Trader por conservar saldo cuando no existen oportunidades adecuadas.

La señal individual de Trader es la siguiente:

$$r_{T,t} = \begin{cases} -b \cdot e_j, & \text{si se cierra } j \text{ y Trader propuso su compra y venta,} \\ -l_t, & \text{si Trader propone una compra que incumple límites,} \\ -R_j, & \text{si Scout marcó } j, \text{ Trader no propuso compra, } R_j > 0 \\ & \text{y la compra era viable,} \\ -h_t, & \text{si termina la selección de artículos, } h_t > 0 \\ & \text{y había oportunidades viables,} \\ 0, & \text{en cualquier otro caso.} \end{cases} \quad (4.8)$$

Trader recibe una penalización por los días adicionales que tarda en cerrar la operación. Si propone una compra inválida, recibe una penalización proporcional a los límites incumplidos. También se le penaliza si no propone comprar un artículo marcado que era rentable y viable. El ROI de esa oportunidad se calcula durante el entrenamiento con los datos históricos, como en la penalización de Scout.

La última penalización evita que Trader invierta una cantidad mínima cuando existen oportunidades adecuadas. Por ejemplo, con $u = 0,50$ y $d = 0,05$, una cartera de 1.000 EUR puede invertir entre 450 EUR y 550 EUR sin que este límite genere penalización. Si solo se invierten 400 EUR y había artículos marcados, rentables y viables para alcanzar 450 EUR, entonces $h_t = \frac{0,45-0,40}{0,45} \simeq 0,11$ y la señal individual de Trader es $-0,11$. Una proporción de 560 EUR sobre 1.000 EUR supera el máximo efectivo del 55 % y se incorpora en l_t .

La señal individual de Portfolio evalúa el cumplimiento de los límites y el rechazo de oportunidades viables:

$$r_{P,t} = \begin{cases} -l_t, & \text{si Portfolio aprueba una compra que incumple límites,} \\ -R_j, & \text{si Scout y Trader propusieron comprar } j, \text{ Portfolio la rechazó, } R_j > 0 \\ & \text{y la compra era viable,} \\ 0, & \text{en cualquier otro caso.} \end{cases} \quad (4.9)$$

Portfolio no recibe una recompensa individual positiva por aprobar una compra permitida. Su incentivo para aprobar operaciones rentables procede de la recompensa cooperativa. Sin embargo, si rechaza una compra que respetaba los límites, existía saldo suficiente y habría terminado con un ROI positivo, recibe una penalización igual a ese ROI. Esta comprobación se realiza únicamente durante el entrenamiento con los datos históricos posteriores. Así, rechazar una operación sigue siendo neutro cuando protege la cartera, pero no se convierte en una forma gratuita de evitar cualquier compra.

La recompensa que recibe cada agente combina la recompensa común, vinculada al resultado de las operaciones y al cumplimiento de los límites, y la señal individual asociada a su responsabilidad concreta:

$$r_t^i = g \cdot r_t^{\text{cartera}} + (1 - g) \cdot r_{i,t}, \quad 0 \leq g \leq 1. \quad (4.10)$$

En la Ecuación 4.10, r_t^{cartera} es la recompensa común definida en la Ecuación 4.3, $r_{i,t}$ es la señal individual del agente i y g determina el peso de cada parte. La implementación utiliza inicialmente $g = 0,70$. Por tanto, el 70 % de la recompensa de cada agente procede del resultado común de la operación y el 30 % de su responsabilidad individual.

Por ejemplo, considérese la venta tardía del ejemplo anterior, con $R_j = 0,10$, $e_j = 4$ y $b = 0,01$. La recompensa cooperativa es 0,02. Las señales individuales de Scout y Portfolio son 0, porque la operación fue rentable y respetó los límites. La señal individual de Trader es $-0,01 \cdot 4 = -0,04$, debido a los días adicionales. Con $g = 0,70$, las recompensas híbridas son $r_t^S = 0,014$, $r_t^T = 0,002$ y $r_t^P = 0,014$. De este modo, los tres agentes comparten el resultado económico, mientras que Trader recibe una penalización adicional por haber demorado el cierre.

Después de cada transición, el entorno devuelve una recompensa híbrida a cada agente. Esta recompensa es cero cuando no se cierra una operación, no se incumple una restricción, no se omite una oportunidad viable y no existe un déficit de inversión penalizable. Durante el entrenamiento, la sucesión de estas recompensas se utiliza para calcular el retorno acumulado de cada política, según la Ecuación 2.6. Por tanto, las recompensas cooperativa e individual no se suman entre agentes. Primero se combinan

para obtener r_t^i y, después, el algoritmo de aprendizaje por refuerzo utiliza esa señal para actualizar la política del agente i .

4.6 Restricciones de riesgo y validez de acciones

Las restricciones se comprueban antes de ejecutar una compra simulada. Su finalidad es impedir que una señal supervisada favorable apruebe una operación que no respeta los límites de la cartera. Dentro del modelo MARL, estas restricciones pertenecen al entorno y no sustituyen a los agentes. El entorno calcula los límites aplicables a partir del estado global, Portfolio recibe parte de esa información en su observación local y la acción conjunta solo se transforma en una compra cuando la propuesta supera esos controles [16, 17].

Esta ubicación es importante porque separa decisión y validez. Scout puede marcar un candidato y Trader puede proponer una compra, pero esas acciones no significan que la operación deba ejecutarse. Portfolio puede aprobar o rechazar la propuesta, y aun así el entorno conserva la comprobación final para garantizar que una acción inválida no modifique la cartera. De este modo, las restricciones actúan como reglas del simulador y como información de aprendizaje: los agentes observan avisos o incumplimientos y reciben penalizaciones cuando insisten en operaciones que no pueden realizarse.

La configuración inicial incluye los siguientes límites, que pueden modificarse en cada experimento:

- **Tamaño de posición:** el importe de una sola compra no puede superar el 20 % del valor neto de la cartera.
- **Exposición por artículo:** la suma de las posiciones del mismo artículo, incluida la compra propuesta, no puede superar el 30 % del valor neto.
- **Exposición por plataforma:** las posiciones adquiridas en una misma plataforma no pueden superar el 70 % del valor neto.
- **Reserva de efectivo:** después de la compra debe permanecer disponible al menos el 10 % del valor neto de la cartera.

Por ejemplo, una compra de 250 EUR en una cartera valorada en 1.000 EUR incumple el límite de tamaño de posición, porque representa el 25 % del valor neto y el máximo permitido es el 20 %. Si el sistema ya mantiene posiciones abiertas del mismo artículo por 200 EUR, una nueva compra de 150 EUR también puede incumplir la exposición por artículo, ya que la exposición conjunta ascendería al 35 %. En ambos casos, la operación puede parecer atractiva por retorno esperado, pero no debe abrirse porque modifica la cartera de una forma incompatible con el nivel de riesgo definido para el experimento.

La banda flexible descrita en la Sección 4.5 se comprueba junto con estos límites. Una propuesta que supera su máximo efectivo, $u + d$, se considera un incumplimiento. Los límites enumerados anteriormente no incorporan tolerancia. Esta diferencia permite combinar dos tipos de control. Por un lado, los límites estrictos impiden operaciones que comprometen demasiado capital o dejan poco saldo disponible. Por otro, la banda

flexible orienta a Trader para que no invierta muy por debajo o por encima de la proporción objetivo cuando existen oportunidades viables.

El periodo de bloqueo de intercambio no constituye un límite adicional, sino una condición de la posición simulada. Una compra puede estar permitida y, aun así, el artículo adquirido no puede venderse hasta que termine ese periodo. El simulador conserva esa información para que Trader y Portfolio no propongan una venta imposible.

En términos de transición, una compra válida reduce el saldo disponible, crea una posición abierta y registra la fecha de desbloqueo. Una compra inválida no cambia la cartera, pero deja constancia del motivo de rechazo. Esa diferencia evita mezclar dos situaciones distintas: no comprar porque los agentes deciden esperar y no comprar porque la operación incumple una regla. En el primer caso la recompensa puede ser cero o depender de una oportunidad perdida; en el segundo se aplica la penalización definida en la Sección 4.5.

Las restricciones también hacen que la señal supervisada tenga un papel acotado. La probabilidad producida por el modelo supervisado puede ayudar a ordenar candidatos, pero no puede aprobar por sí sola una operación. El entorno siempre evalúa el estado de cartera en el instante actual, incluyendo saldo, capital bloqueado y exposiciones acumuladas. Por tanto, dos candidatos con la misma probabilidad pueden recibir decisiones diferentes si uno encaja en la cartera y el otro supera los límites de riesgo.

Entorno y preparación de datos

Este capítulo describe el entorno de datos empleado en los experimentos. En primer lugar, identifica las fuentes de información disponibles y el papel que desempeña cada una en la recogida de datos de los mercados secundarios de artículos de CS2. A continuación, explica cómo se normalizan y validan los registros para conservar su procedencia, su fecha de captura y los atributos que identifican cada artículo.

El capítulo presenta después las variables construidas a partir del histórico y el procedimiento temporal con el que se forman los conjuntos de entrenamiento, validación y prueba. Esta preparación permite que los modelos utilicen únicamente la información disponible en cada fecha y que sus resultados se evalúen sin incorporar información futura.

5.1 Fuentes de datos

El Capítulo 3 describe los mercados secundarios que delimitan el caso de estudio. Este apartado concreta el uso de cada fuente dentro del flujo de datos. El flujo principal comienza con SteamDT y continúa con la consulta de Steam Community Market y BUFF para los artículos seleccionados:

- **SteamDT** genera una lista inicial de artículos candidatos [19]. Esta lista reduce el número de artículos que deben consultarse en las fuentes de precio.
- **Steam Community Market** proporciona datos actuales e históricos de precio, órdenes de compra y volumen de intercambios [3]. Su histórico se utiliza para construir el dataset temporal.
- **BUFF** proporciona el precio actual de compra y las órdenes de compra con los que se contrasta una operación de compraventa entre plataformas [7]. No se utiliza como fuente histórica principal del entrenamiento.
- **Reconocimiento óptico de caracteres (OCR) e importación manual:** son dos vías complementarias al flujo principal. El OCR extrae texto y cifras de una captura de pantalla. La importación manual permite cargar una tabla o un fichero preparado previamente. Ambas vías se emplean cuando los datos no están disponibles mediante una consulta automatizada.

Cada registro almacenado conserva la fuente, la fecha de captura, la moneda original y los atributos que identifican el artículo. La siguiente sección describe cómo se normalizan y validan estos campos. Los procedimientos técnicos de captura se presentan en el Capítulo 6.

5.2 Normalización y validación de los datos

Una fuente puede expresar los mismos datos con nombres, monedas, fechas o formatos distintos. Para evitar que esas diferencias se interpreten como diferencias de mercado, cada captura se transforma en un **registro de mercado**: una fila de dato que junta un artículo, la fuente de la que procede, la fecha de captura de datos y los valores publicados en ese momento. Este registro es la unidad que se utiliza después para construir variables temporales.

La normalización conserva el nombre y los atributos que identifican el artículo, como su acabado, nivel de desgaste, *float* o StatTrak, y los asocia a una misma identificación interna. También permite comparar precios publicados en monedas distintas. Por ejemplo, si Steam Community Market muestra el precio de un artículo en euros y BUFF lo muestra en yuanes, el sistema convierte ambos precios a una moneda de comparación. Al mismo tiempo, conserva el precio y la moneda publicados por cada fuente para poder comprobar posteriormente de dónde procede cada valor. Esta conversión solo permite comparar precios. Las comisiones, el beneficio neto y la rentabilidad se calculan después mediante las reglas económicas definidas en el Capítulo 4.

La fecha es igualmente necesaria. Los registros se ordenan por fecha de captura para que una variable calculada en un día solo emplee datos disponibles hasta ese día. Si una fuente proporciona históricos, cada punto histórico mantiene su propia fecha en lugar de recibir la fecha de descarga. De este modo, el conjunto temporal distingue cuándo se publicó un dato de cuándo el sistema lo incorporó.

Antes de almacenar un registro, el sistema comprueba que el artículo pueda identificarse y que los campos necesarios para su tipo de dato estén presentes. Por ejemplo, un precio histórico sin fecha o sin valor numérico no forma parte de la serie temporal. Asimismo, un mismo artículo, fuente, fecha y tipo de dato, como precio, orden de compra o volumen de intercambios, no se incorpora dos veces. Los datos obtenidos mediante OCR deben superar además sus controles específicos de lectura y confianza, explicados en el Capítulo 6. Los detalles de captura, persistencia y deduplicación se describen también en ese capítulo.

5.3 Variables y conjunto temporal

Los registros normalizados no se emplean directamente para entrenar los modelos. A partir de ellos se construyen **variables de entrada**, es decir, valores numéricos que resumen la situación de un artículo en una fecha determinada. Todas las variables de una fila se calculan únicamente con los datos disponibles hasta esa fecha. Por tanto, el resultado posterior de una posible operación no forma parte de la información utilizada para predecirla.

Las variables de mercado se agrupan en los siguientes bloques:

- **Precios y comparación entre mercados:** precio de venta publicado en Steam Community Market y en BUFF, precio de las órdenes de compra de BUFF y diferencias entre esos precios antes y después de aplicar las comisiones correspondientes.

- **Actividad del mercado:** volumen de intercambios registrado en Steam Community Market y número de ofertas publicadas en BUFF. Estos valores indican cuánta actividad se ha producido alrededor del artículo.
- **Evolución temporal:** cambios y retornos de precio a 1, 3 y 7 días, así como la media y la desviación de los últimos 7 días. También se incorporan el día de la semana, el mes y un indicador de fin de semana.

Los valores derivados de la comparación de precios en la fecha inicial, como el beneficio neto y el ROI calculados con los precios disponibles en esa fecha, pueden utilizarse como variables de entrada. No se incorporan al conjunto de datos las variables de cartera, porque el simulador las crea y actualiza durante cada episodio, tal como se describe en el Capítulo 4.

Para formar el conjunto temporal, se toma un artículo i y una fecha t . El sistema calcula sus variables con los registros disponibles hasta t y busca después el primer registro futuro de precio dentro del intervalo comprendido entre $t + 8$ y $t + 15$ días. El primer límite representa el periodo de bloqueo de intercambio utilizado en la simulación. Los siete días adicionales permiten conservar el ejemplo cuando la fuente no publica un registro exactamente en el octavo día.

La etiqueta $y_{i,t}$ vale 1 cuando la operación simulada con ese registro futuro produce un beneficio neto no negativo y un ROI de al menos el 10 %. En cualquier otro caso, vale 0. Si d representa la fecha del primer registro futuro encontrado, $B_{i,d}$ el beneficio neto calculado y $R_{i,d}$ el ROI, la etiqueta se define como:

$$y_{i,t} = 1 \iff B_{i,d} \geq 0 \quad \wedge \quad R_{i,d} \geq 0,10.$$

Esta etiqueta no predice una simple subida de precio. Indica si, bajo las reglas económicas del Capítulo 4, una compra en la fecha t habría alcanzado el criterio económico definido al llegar la fecha d .

El conjunto actual separa los datos por fecha para evitar que los modelos conozcan información futura:

- **Entrenamiento:** desde el 2 de julio de 2025 hasta el 16 de diciembre de 2025.
- **Validación:** desde el 1 de enero de 2026 hasta el 13 de febrero de 2026.
- **Prueba:** desde el 1 de marzo de 2026 hasta el 30 de junio de 2026.

Los quince días anteriores a cada frontera se excluyen del conjunto. Esta franja de purga impide que un registro de entrenamiento o validación utilice como resultado un precio perteneciente al periodo siguiente. Con este procedimiento, cada conjunto conserva únicamente la información disponible en su periodo y permite evaluar los modelos sin incorporar datos futuros.

Implementación y operación

Este capítulo describe la implementación del sistema propuesto. Explica cómo se obtienen y almacenan los datos de los mercados, cómo se construyen los conjuntos temporales para los modelos y cómo se conectan la predicción supervisada, el simulador de cartera y el entorno multiagente. También presenta la consola local utilizada para consultar oportunidades, ejecutar tareas de captura y revisar el estado de los componentes.

La implementación mantiene separadas la adquisición de datos, la persistencia, la preparación de conjuntos, los modelos y la interfaz de consulta. Esta separación permite repetir un experimento, localizar el origen de un dato y revisar una oportunidad sin ejecutar compras ni ventas reales. La estructura detallada del código se incluye en el Anexo A.1.

6.1 Componentes y responsabilidades

La implementación se organiza en componentes que intercambian datos mediante formatos definidos. Cada componente recibe campos concretos y produce un resultado con campos conocidos por el componente siguiente. Por ejemplo, un extractor no entrega directamente una recomendación, entrega un registro de mercado con el artículo, la fuente, la fecha de captura, el precio, la moneda, el volumen, el origen y la calidad de la lectura. La persistencia conserva esa información y los módulos posteriores construyen las variables o ejecutan los modelos que correspondan. Esta secuencia permite localizar un error sin repetir todo el flujo.

- **Adquisición:** localiza artículos, consulta las fuentes y convierte cada resultado en un registro de mercado trazable.
- **Preparación y persistencia:** conserva el estado más reciente y el histórico incremental. A partir de este último crea los conjuntos temporales descritos en el Capítulo 5.
- **Predicción supervisada:** recibe las variables preparadas a partir del histórico y calcula una probabilidad para cada posible operación. Esta probabilidad es una señal de entrada, no una decisión.
- **Simulación y entorno multiagente:** recibe los datos de mercado, el estado de cartera y la señal supervisada. Los roles proponen y validan acciones, mientras el simulador aplica las reglas económicas y calcula las consecuencias de cada decisión simulada.
- **Consola local:** permite ejecutar tareas controladas y revisar oportunidades, sesiones y resultados sin enviar órdenes a los mercados.

La predicción tabular se implementa con scikit-learn [22]. Cada modelo entrenado se guarda con Joblib junto con un fichero que identifica sus variables de entrada, su configuración y las métricas obtenidas durante el entrenamiento. El entorno multiagente dispone de una interfaz PettingZoo [23] y de una integración con Ray/RLlib [24]. PyTorch proporciona los componentes de cálculo empleados durante el entrenamiento [25]. Las pruebas unitarias y de integración se ejecutan con Pytest, una herramienta de Python para localizar y ejecutar pruebas automatizadas, y con pytest-asyncio para los casos asíncronos [26]. Ruff revisa estilo y posibles errores estáticos, mientras que Mypy comprueba las anotaciones de tipos antes de ejecutar el código [27, 28]. Estas herramientas materializan la propuesta, pero no sustituyen la justificación metodológica de los capítulos anteriores.

6.2 Adquisición y persistencia de datos

Antes de capturar datos de una fuente que exige autenticación, la persona usuaria debe iniciar sesión desde la pantalla de sesiones del navegador. Se ha desarrollado un proceso que abre un navegador visible para que el usuario pueda completar ese paso y conserva localmente el estado de la sesión una vez finalizado. Así, las ejecuciones posteriores pueden reutilizar la sesión mientras siga vigente. Las credenciales y el estado de navegador no se guardan en el repositorio ni en el archivo de configuración.

Playwright [29] es la biblioteca que controla ese navegador y permite consultar las fuentes que necesitan navegación. Se utiliza tanto para abrir el navegador durante el inicio de sesión como para realizar las capturas posteriores con la sesión local disponible.

Cada trabajo de scraping registra inicio, conectores utilizados, reintentos, progreso, fichero de log y resultado final. Los límites de concurrencia, el tamaño de lote y el tiempo de espera son configurables. El objetivo no es ejecutar el mayor número de peticiones posible, sino mantener un proceso que pueda detenerse, reanudarse y auditarse si una fuente cambia su interfaz.

Algoritmo 3 Flujo de scraping.

Entrada: Artículos localizados, conectores activos y configuración de trabajo

Salida: Registros de mercado persistidos y estado de ejecución

- 1: Crear identificador de trabajo y registrar el inicio
 - 2: **for all** lotes de candidatos **do**
 - 3: **for all** conectores habilitados **do**
 - 4: Consultar la fuente con sesión, límite y espera configurados
 - 5: Normalizar precio, moneda, volumen y fecha de captura
 - 6: Validar el resultado y registrar anomalías o reintentos
 - 7: **end for**
 - 8: Persistir registros de mercado válidos y actualizar el progreso
 - 9: **end for**
 - 10: Registrar resultado, cobertura y referencia del log
-

La base de datos almacena el estado actual en `market_items` y las series en `market_history_points`. Guardar el histórico en formato largo permite incorporar nuevas métricas sin alterar el esquema de una tabla ancha. Cada registro de mercado

incluye el origen y la fecha, por lo que una variación de precio puede relacionarse con la fuente y con la ejecución que la recogió.

La deduplicación se realiza antes de persistir y las pruebas de integración verifican inserciones, actualizaciones y registros repetidos. Cada prueba revierte su transacción al terminar, de modo que las comprobaciones no alteran el histórico utilizado en los experimentos.

El OCR se incorpora como vía complementaria de adquisición. Convierte capturas, tablas o interfaces no estructuradas en registros de mercado equivalentes a los obtenidos mediante una consulta estructurada. Aunque esta vía es más frágil que una fuente estructurada, permite incorporar datos cuando una plataforma no ofrece un acceso automatizable o cuando se dispone de material histórico en forma de imagen.

La implementación actual se centra en tablas y mensajes de mercado, cuyas filas presentan una estructura regular. El flujo aplica cuatro controles:

- **Preprocesado:** OpenCV [30] amplía la imagen por un factor dos, la convierte a escala de grises, mejora el contraste local con CLAHE [31], reduce ruido y aplica una binarización adaptativa. CLAHE divide la imagen en zonas, ajusta el contraste de cada una y limita ese refuerzo para no amplificar el ruido.
- **Reconocimiento:** Tesseract identifica las palabras de la captura y proporciona una confianza para cada una [32].
- **Interpretación:** el parser normaliza espacios y separadores decimales, e identifica el artículo, su calidad, la plataforma, el precio, la moneda y el volumen cuando aparecen en una fila.
- **Validación:** solo se conserva un registro de mercado cuando su confianza es suficiente y el precio es positivo y menor de un millón de unidades monetarias.

La confianza agregada es la media de las n palabras cuya confianza es válida, donde c_j representa la confianza de la palabra j :

$$c_{\text{OCR}} = \frac{1}{100n} \sum_{j=1}^n c_j.$$

Solo se conserva un registro de mercado cuando $c_{\text{OCR}} \geq 0,5$ y el precio supera los controles descritos. Esta regla evita que una captura dudosa entre en el histórico sin quedar identificada.

Las importaciones manuales y CSV complementan el flujo cuando se necesitan muestras controladas, fixtures o históricos parciales sin depender de una sesión de navegador. Esta vía se utiliza para validar parsers, modelos de persistencia y reglas económicas antes de automatizar el proceso completo.

6.3 Entrenamiento y ejecución de modelos

Los conjuntos temporales preparados en el Capítulo 5 alimentan los procesos de entrenamiento. Los modelos se ajustan con el conjunto de entrenamiento, su configuración se selecciona mediante validación y el conjunto de prueba queda reservado para la evaluación final descrita en el capítulo de experimentación. Esta separación mantiene independientes la selección del modelo y la medición de su resultado final.

El módulo supervisado entrena modelos tabulares y guarda el fichero del modelo seleccionado junto con el listado de variables de entrada, la configuración empleada y un informe de entrenamiento. En una consulta posterior, el servicio de inferencia comprueba que recibe esas mismas variables y devuelve una probabilidad junto con la versión del modelo. Esa probabilidad pasa al simulador para evaluar la posible operación, pero no ejecuta una compra.

El simulador de cartera se mantiene separado de los modelos de aprendizaje. Implementa las comisiones, el bloqueo de intercambio, las posiciones y las restricciones explicadas en el Capítulo 4. El entorno multiagente reutiliza ese simulador para ejecutar episodios y conserva, para cada paso, las observaciones locales, las acciones propuestas, el resultado de la transición y el desglose de las recompensas. De este modo, el entrenamiento puede revisar por qué una política recibe una recompensa sin alterar los datos de mercado ni enviar una orden real.

El entrenamiento MARL sigue el esquema de entrenamiento centralizado y ejecución descentralizada descrito en la Sección 2.4. Durante el entrenamiento, cada actor recibe únicamente la observación correspondiente a su rol, mientras que el modelo evaluador central utiliza el estado compartido para valorar la decisión conjunta. Cuando se carga un modelo entrenado para ejecutar una simulación o generar una recomendación, se utilizan solo los tres actores locales. El modelo evaluador central no interviene en esa ejecución.

Los comandos de construcción de conjuntos de datos y de entrenamiento conservan, sin secretos, la configuración de cada ejecución, el rango temporal, las variables, el tamaño de cada partición, el porcentaje de casos que cumplen y no cumplen el criterio económico, los hiperparámetros y las métricas obtenidas. Esta información permite distinguir dos situaciones que visualmente podrían parecer iguales: ejecutar un modelo con más datos y repetir exactamente una configuración anterior. La evaluación final sobre el conjunto de prueba y la comparación entre configuraciones se presentan en el capítulo de experimentación.

Las pruebas automatizadas verifican los componentes de dominio, persistencia, OCR, scraping, simulación, MARL y panel web antes de utilizar sus resultados en tablas y gráficas.

6.4 Consola local de consulta

La consola local convierte el estado técnico en una interfaz de revisión. Su pantalla principal separa la lista de oportunidades del detalle de la oportunidad seleccionada. La lista muestra el nombre del artículo, la secuencia prevista de compra y venta, los precios publicados y el beneficio neto estimado. El detalle indica la probabilidad calculada por el modelo supervisado, las fechas de captura de los datos y las restricciones de cartera que intervienen.

El panel no ejecuta compras. Las acciones de abrir Steam o BUFF sirven para que la persona usuaria revise la oportunidad en su fuente. Los estados *revisar*, *observar* y *bloqueado* representan una decisión explicable, no una orden automática. Esta restricción es importante mientras la validación estadística y multiagente sigue en fase exploratoria.

Revisar La información disponible y el resultado económico estimado justifican una comprobación manual.

Observar Faltan datos o conviene esperar una nueva captura antes de evaluar la posible operación.

Bloqueado Los costes, el riesgo o las restricciones de cartera impiden continuar con la posible operación.

La consola se organiza en las siguientes vistas:

- **Oportunidades:** muestra los artículos candidatos, sus precios, el beneficio neto estimado y el estado de revisión.
- **Captura:** permite iniciar una obtención manual de datos y consultar su progreso, los conectores utilizados y el resultado.
- **Sesiones:** informa de la vigencia de las sesiones locales de Steam y BUFF necesarias para las fuentes que requieren autenticación.
- **Modelo:** identifica la versión cargada, el conjunto de datos utilizado, las métricas disponibles y su carácter experimental.
- **Agentes y cartera:** muestra los roles MARL, el capital bloqueado, los límites aplicados y las propuestas rechazadas.

Experimentación y análisis

Este capítulo presenta la evaluación experimental del sistema desarrollado. La experimentación no se limita a comparar modelos, sino que comprueba de forma progresiva las piezas necesarias para que una decisión simulada sea interpretable: reglas económicas, construcción de datos, modelos supervisados, reconocimiento mediante OCR, simulador de cartera y entorno multiagente. Esta organización permite separar las verificaciones técnicas de los resultados de aprendizaje, evitando que una prueba de funcionamiento se interprete como una mejora económica demostrada.

El capítulo se estructura por fases. Primero se validan las reglas y los datos que sostienen el simulador. Después se analizan los modelos supervisados y los cortes temporales de compraventa Steam/BUFF. A continuación se presentan las comprobaciones funcionales del entorno MARL y la matriz de entrenamiento multiagente. Finalmente, se delimita la evaluación final reservada y se discuten las limitaciones que condicionan la interpretación de los resultados.

7.1 Diseño experimental

Para la primera fase de captura de datos, se ha construido un scraper, es decir, un programa que consulta fuentes web y transforma sus respuestas en datos estructurados, junto con persistencia de registros, sesiones y cookies de sesión, pequeños datos locales que permiten reutilizar una autenticación ya iniciada. Esta parte no se evalúa con una métrica de rentabilidad, pero condiciona todos los experimentos posteriores: sin datos trazables, fechas de captura y sesiones reproducibles no se puede construir un histórico.

Durante esta fase también se han observado restricciones externas. Algunas cuentas o sesiones han sufrido bloqueos temporales y ha sido necesario ajustar el tamaño de los lotes, los reintentos y los retrasos entre peticiones. Por otra parte, las plataformas han sufrido cambios estructurales y ha sido necesario adaptar el código para mantener la compatibilidad. Por estos motivos, la evaluación no presenta la captura como un proceso instantáneo, sino como una parte del desarrollo que obliga a equilibrar cobertura de datos, estabilidad de sesión y respeto de los límites operativos de cada plataforma. Después de esta base se migran reglas económicas, se construyen conjuntos supervisados, se crea el conjunto de compraventa Steam/BUFF y se entrena la matriz multiagente.

El criterio de evaluación no es solo predictivo. Una señal puede tener buena precisión y aun así no ser operativa si aparece pocas veces, si depende de artículos concretos o si no supera costes de transacción. Por eso las métricas del modelo supervisado se interpretan como apoyo a la decisión, no como resultado económico final. Esta distinción es especialmente importante en el conjunto Steam/BUFF, donde el objetivo no es anticipar una subida de precio aislada, sino estudiar una posible compraventa con precios normalizados, comisiones y horizonte temporal.

En la parte supervisada se comparan modelos tabulares como regresión logística, random forest e histogram gradient boosting, familias habituales en clasificación tabular [33, 34]. Estos modelos se contrastan con una referencia simple que predice la clase mayoritaria, de modo que sus resultados puedan interpretarse frente a la tasa base del conjunto. En cambio, la parte multiagente no compara clasificadores, sino políticas que actúan dentro de un simulador de cartera. Por eso se separan primero los controles funcionales del entorno, que validan compra, bloqueo, venta y trazabilidad, y después la matriz de entrenamiento, que compara configuraciones de aprendizaje con semillas distintas sobre el bloque de validación, sin utilizar el conjunto de prueba.

El entrenamiento multiagente se realiza con PPO, *Proximal Policy Optimization*, un algoritmo de aprendizaje por refuerzo que ajusta una política a partir de episodios simulados y limita el tamaño de cada actualización para evitar cambios bruscos entre una iteración y la siguiente [35]. En este trabajo no se entrena una única política general: se mantiene una política por rol, Scout, Trader y Portfolio. Cada política recibe sus observaciones locales, selecciona acciones y se actualiza con las recompensas registradas por el entorno.

Algoritmo 4 Entrenamiento con PPO

Entrada: Entorno paralelo, políticas Scout, Trader y Portfolio, iteraciones y pasos

Salida: Modelo guardado (checkpoint) y métricas de la ejecución

- 1: Inicializar una política PPO por rol con observaciones vectoriales
 - 2: **for** cada iteración de entrenamiento **do**
 - 3: Ejecutar episodios y registrar $(o_t^i, a_t^i, R_t, m_t^i)$
 - 4: Estimar ventajas $\hat{A}_t$ a partir de las transiciones registradas
 - 5: Optimizar cada política con el objetivo recortado de PPO
 - 6: **end for**
 - 7: Evaluar el modelo guardado sin actualizar sus parámetros
 - 8: Guardar efectivo, posiciones, operaciones y recompensa media
-

7.2 Validación técnica y pruebas

Las pruebas técnicas no se usan para demostrar rentabilidad, sino para comprobar que los resultados experimentales proceden de componentes que funcionan de forma controlada. Antes de interpretar un modelo supervisado o una política MARL, se valida que las reglas económicas, la captura de datos, la persistencia, la construcción de conjuntos temporales, la inferencia supervisada, el simulador de cartera y el entorno multiagente producen salidas coherentes. Esta separación es importante porque un resultado numérico puede parecer válido aunque se haya calculado sobre datos mal normalizados, fechas desplazadas, comisiones incorrectas o acciones imposibles dentro del simulador.

La validación se organiza en tres capas. La primera comprueba funciones aisladas: conversión de moneda, cálculo de comisiones, bloqueo temporal, límites de cartera, extracción de campos de mercado y construcción de etiquetas. La segunda revisa flujos completos sin depender de una ejecución real de compraventa: captura, normalización, almacenamiento, creación de datasets y carga de modelos. La tercera comprueba que el entorno multiagente puede recorrer episodios de compra, mantenimiento, venta y

rechazo de operaciones, registrando observaciones, acciones, recompensas y estado de cartera.

El conjunto de pruebas automatizadas combina tres tipos de comprobación ya descritos en el Capítulo 6: ejecución de casos con Pytest, revisión estática con Ruff y comprobación de tipos con Mypy. En esta sección no se valoran por la herramienta empleada, sino por el riesgo que reducen. Las pruebas unitarias comprueban el comportamiento determinista del simulador, las reglas económicas del procedimiento manual, el OCR, los conectores de mercado, los datasets supervisados, el dataset de trading, el servicio de scoring y los componentes MARL. La prueba de integración de persistencia se mantiene separada porque necesita una base de datos disponible.

Bloque	Riesgo controlado
Reglas y cartera	Cálculos económicos incorrectos o posiciones imposibles.
Datos y captura	Registros incompletos, mal normalizados o sin trazabilidad temporal.
Modelo supervisado	Variables incompatibles, modelos no cargables o inferencia incoherente.
Simulación MARL	Acciones inválidas, recompensas mal asignadas o estado de cartera inconsistente.
Interfaz y operación	Comandos inseguros, estados poco claros o recomendaciones no revisables.

Tabla 7.1: Cobertura de pruebas.

Las pruebas de **rendimiento técnico** se interpretan con cautela. En este proyecto el rendimiento no se reduce al tiempo de ejecución: también importan la cobertura de datos, el número de señales generadas, el coste de entrenamiento, el capital bloqueado, el **drawdown** y las operaciones rechazadas. Las ejecuciones mínimas de entrenamiento y los episodios MARL registran estas métricas para comprobar que el sistema produce salidas interpretables y comparables entre configuraciones. Aun así, estas pruebas no sustituyen la evaluación económica final: validan que el sistema puede medir, no que una estrategia sea mejor que otra.

7.3 Métricas

Las métricas permiten interpretar el sistema desde varias perspectivas. En este trabajo no basta con saber si un modelo produce una señal favorable, porque una señal puede ser estadísticamente correcta y aun así no ser útil para operar. Por ejemplo, puede aparecer muy pocas veces, puede depender de un conjunto pequeño de artículos, puede que el beneficio no superar las comisiones o puede exigir un capital que la cartera no tiene disponible. Por ese motivo, la evaluación separa la calidad predictiva, el resultado económico, el riesgo asumido y el rendimiento técnico del sistema.

- **Métricas predictivas del modelo supervisado.** Estas métricas se aplican al modelo que produce la señal supervisada. Su objetivo es medir si la estimación histórica aporta información antes de que intervenga el entorno MARL. La precisión indica qué proporción de las señales marcadas como favorables termina

siendo realmente favorable. La **exhaustividad**, también denominada *recall*, mide qué proporción de los casos favorables detecta el modelo. La métrica F1 combina ambas para evitar mirar solo una de ellas. También se emplean ROC-AUC y PR-AUC, aunque la segunda resulta más informativa cuando la clase positiva domina o cuando hay pocos ejemplos negativos [36]. Además, el **Brier score** mide el error de las probabilidades estimadas y la **calibración de probabilidades** comprueba si una probabilidad del modelo se corresponde con la frecuencia observada [37, 38]. En el uso operativo se presta especial atención a la precisión en umbrales altos, porque una señal menos frecuente pero más fiable puede ser más útil que muchas señales ruidosas.

- **Métricas económicas de la operación.** Estas métricas miden el resultado económico de una compra, una venta o una secuencia de decisiones. Incluyen el beneficio neto, el retorno porcentual, el saldo final, el valor de la cartera y el número de señales operativas. El beneficio neto descuenta precio de compra, comisiones, conversión de divisa y costes aplicables; por tanto, es la medida más directa de si una operación concreta aporta ganancias. El retorno porcentual permite comparar operaciones de distinto tamaño, porque relaciona el beneficio con el capital empleado. El saldo final y el valor de cartera son necesarios cuando la evaluación incluye posiciones que todavía no se han cerrado, ya que una parte del resultado puede estar en efectivo y otra parte en activos pendientes de venta.
- **Métricas de riesgo y uso de cartera.** Estas métricas responden a una pregunta distinta: si una estrategia es asumible aunque parezca rentable. Se consideran el capital bloqueado medio, el número de posiciones abiertas, la exposición por activo o plataforma, las operaciones rechazadas por riesgo y el impacto del *trade hold*. También se usa el **drawdown**, entendido como la mayor caída del valor de la cartera respecto a un máximo anterior. Estas medidas son importantes porque el sistema no opera con capital infinito. Una política que concentra demasiadas compras en un único artículo, deja mucho saldo inmovilizado o depende de activos bloqueados puede ser mala aunque algunas operaciones individuales tengan beneficio.
- **Métricas de rendimiento técnico y volumen.** El rendimiento técnico recoge cuánto tarda el sistema en ejecutar sus procesos y qué volumen de información maneja. En esta memoria se registra el tiempo de ejecución de las pruebas automatizadas, el número de pruebas superadas, el número de filas de cada partición, el número de artículos cubiertos, las iteraciones de entrenamiento, los pasos de entorno y los episodios simulados. Para la captura de datos, el tiempo no depende solo del código: también influyen los límites de las plataformas, los bloqueos temporales, la reutilización de cookies de sesión, los reintentos y los retrasos introducidos para no saturar las fuentes. Por eso, cuando se habla de cuánto tarda la captura, debe interpretarse como rendimiento operativo bajo restricciones externas, no como una medida de velocidad local.

Esta separación de métricas evita mezclar conclusiones. Las métricas predictivas indican si la señal supervisada es razonable; las métricas económicas indican si una decisión produce beneficio; las métricas de riesgo indican si la cartera puede asumir esa

decisión; y las métricas de rendimiento técnico indican si el sistema se puede ejecutar, repetir y auditar con los recursos disponibles.

7.4 Resultados

Esta sección reúne los resultados obtenidos durante la construcción del sistema. Se presentan primero las comprobaciones sobre reglas y modelos y después las pruebas con datos de trading, OCR y agentes cooperativos.

7.4.1 Migración de reglas económicas

La primera fase ha consistido en convertir reglas del procedimiento manual en funciones comprobables. Se han migrado conversiones EUR/CNY, factores de comisión, cálculo de beneficio neto, rentabilidad porcentual, fecha de desbloqueo y capital bloqueado. Esta fase no produce un resultado de rentabilidad por sí misma, pero es necesaria para que los experimentos posteriores midan el mismo dominio que se utilizaba manualmente.

El resultado principal de esta fase es metodológico: las reglas económicas pasan de estar dispersas en fórmulas de hoja de cálculo a estar encapsuladas en módulos de simulación, con pruebas unitarias y parámetros explícitos. Esto reduce errores silenciosos y permite repetir un experimento con la misma configuración.

7.4.2 Resultados del modelo supervisado

La fase supervisada conserva una exploración de dirección Steam con 47.467 filas de entrenamiento, 19.252 de validación y 18.412 de prueba. Su etiqueta indica dirección de precio, no beneficio neto entre plataformas. Se mantiene como evidencia de que el pipeline de variables, calibración y selección funciona con una muestra de mayor tamaño, pero no se utiliza para aprobar compras. El protocolo principal de este capítulo es el corte BUFF–Steam que empieza en diciembre de 2025.

Umbral	Precisión	Recall	Señales
0,50	0,716	0,116	1.293
0,70	0,856	0,019	180
0,80	0,947	0,011	95
0,85	0,962	0,006	53
0,90	1,000	0,003	22

Tabla 7.2: Umbrales de dirección.

El umbral 0,85 deja 53 señales de las 18.412 filas de prueba. Esto ilustra el intercambio entre precisión y cobertura: elevar el umbral selecciona menos casos y no convierte la probabilidad en una orden. La calibración se revisa porque una probabilidad solo es útil si representa de forma razonable la frecuencia observada [37, 38]. Estos resultados no se consideran definitivos. Sí indican que el pipeline de entrenamiento, calibración, selección por umbral y evaluación temporal funciona.

7.4.3 Evolución de los datasets de trading

El dataset de compraventa no apareció en una única ejecución. Se han construido varios cortes porque cada uno reveló una limitación diferente del histórico disponible. El primer `trading_profit_v1` reunió 2.331 filas y 50 artículos. Sirvió para comprobar el constructor de etiquetas, las comisiones y la persistencia de características, pero la dirección analizada solo conservaba un caso positivo en validación y otro en prueba. Por ese motivo sus métricas no se comunican como resultado de un modelo.

Después se corrigieron el parser de BUFF, la normalización de moneda y el backfill de puntos históricos. Con estas correcciones se ejecutaron tres cortes BUFF–Steam desde diciembre de 2025. La Tabla 7.3 deja visibles tanto las muestras que permiten una exploración como las que se descartan. Esta distinción es importante: un dataset grande no es suficiente si sus bloques de validación solo contienen una clase.

Corte	Entrenamiento	Validación	Prueba
Inicial	2.331	—	—
Marzo	171	76	95
Mayo	361	76	95
Reciente	551	76	84

Tabla 7.3: Cortes de datos.

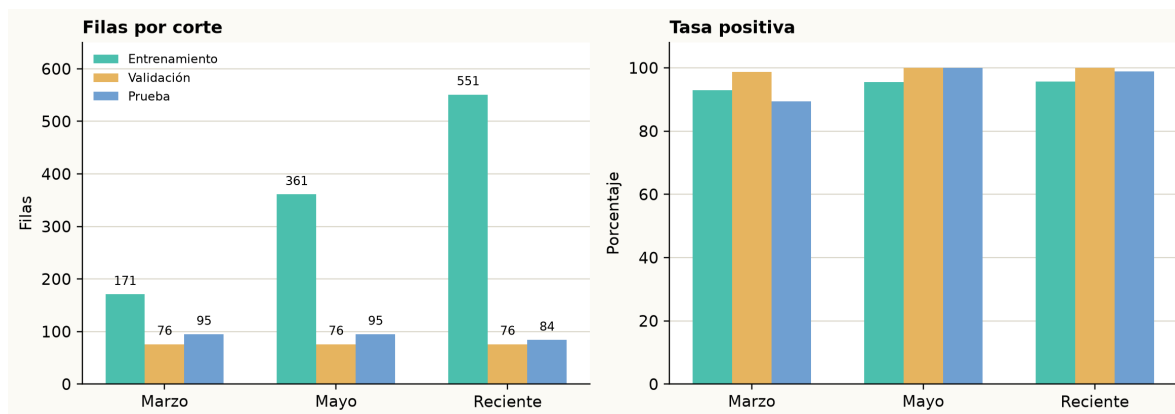


Figura 7.1: Evolución de cortes.

La secuencia mejora la cobertura temporal, pero no el equilibrio de etiquetas. El conjunto inicial contiene 50 artículos, pero solo conserva un caso positivo en validación y otro en prueba. En marzo, la proporción de casos positivos es 93,0 % en entrenamiento, 98,7 % en validación y 89,5 % en prueba. Es el único corte que contiene ambas clases en los tres bloques, aunque sigue muy concentrado en oportunidades positivas.

Los cortes de mayo y reciente amplían el número de filas. En mayo, validación y prueba contienen únicamente casos positivos. En el corte reciente las proporciones son 95,6 %, 100 % y 98,8 %, respectivamente. Estas muestras no habilitan ROC-AUC ni calibración sobre los bloques finales, por lo que se conservan como diagnósticos de calidad de datos y no como experimentos de rendimiento.

El flujo reproducible queda definido así: capturar observaciones, normalizar artículo y moneda, construir la etiqueta a ocho días, eliminar la zona de purga, separar por fecha

y revisar la distribución antes de entrenar. Solo cuando cada bloque incluye resultados favorables y desfavorables se entrena un clasificador. Esta comprobación previa evitó presentar un entrenamiento trivial como una mejora del sistema.

7.4.4 Conjunto actual de compraventa

Las exploraciones anteriores se conservan porque documentan la evolución de la captura y la construcción de etiquetas. Sin embargo, el entrenamiento multiagente actual utiliza `trading_profit_v2`, generado el 25 de agosto de 2026. Este conjunto contiene 36.505 ejemplos de la ruta de compra en BUFF y venta posterior en Steam, con un horizonte de ocho días, tolerancia de siete días para localizar el precio futuro y una purga de quince días entre particiones.

La etiqueta vale uno cuando la operación habría superado un retorno sobre la inversión del 10 % después de aplicar las comisiones configuradas. La Tabla 7.4 resume las particiones. A diferencia de los cortes iniciales, los tres bloques contienen ejemplos rentables y no rentables, aunque persiste una proporción elevada de operaciones rentables en validación y prueba.

Partición	Ejemplos	Artículos	Periodo	Rentables
Entrenamiento	16.274	353	02/07/2025–16/12/2025	64,9 %
Validación	4.394	352	01/01/2026–13/02/2026	81,0 %
Prueba	12.971	354	01/03/2026–30/06/2026	82,5 %

Tabla 7.4: Particiones temporales de `trading_profit_v2`.

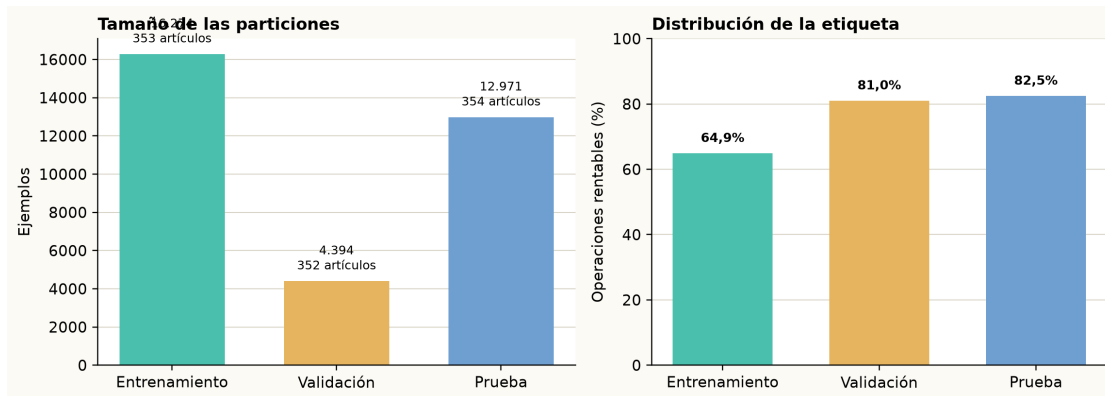


Figura 7.2: Tamaño y distribución de la etiqueta en `trading_profit_v2`.

La separación por fechas mantiene la prueba aislada mientras se seleccionan los parámetros. Por tanto, la validación se utiliza para comparar políticas y detener el entrenamiento, mientras que la prueba queda reservada para una evaluación final de la configuración seleccionada.

7.4.5 Exploración supervisada de marzo

La validación se repitió el 10 de agosto de 2026 con los datos disponibles desde el 1 de diciembre de 2025. Cada ejemplo representa la posibilidad de comprar un artículo

en BUFF y venderlo en Steam tras un horizonte de ocho días. La etiqueta vale uno cuando, después de aplicar las comisiones configuradas, el retorno supera el 10 %. El precio de compra se toma de BUFF y el precio de venta de Steam se valora con el factor de venta de 0,87 empleado por el simulador.

El orden temporal se conserva en todas las fases. Para el corte de marzo se usaron 171 ejemplos de entrenamiento, 76 de validación y 95 de prueba. El entrenamiento termina el 20 de enero, la validación ocupa del 2 al 20 de febrero y la prueba comprende del 4 al 28 de marzo. Se eliminan ocho días antes de cada frontera para que el horizonte futuro de una observación no alcance el bloque siguiente. Ningún parámetro se modifica al consultar la prueba. Esta separación evita mezclar pasado y futuro, una precaución esencial cuando las observaciones tienen dependencia temporal [39].

La Figura 7.3 hace visible una limitación que no se aprecia al mirar solo el número de filas. Los tres bloques contienen las mismas 19 representaciones de artículo y están dominados por la clase rentable. La separación temporal evita que se use información futura, pero no demuestra por sí sola que el modelo generalice a artículos que no ha visto durante el entrenamiento.

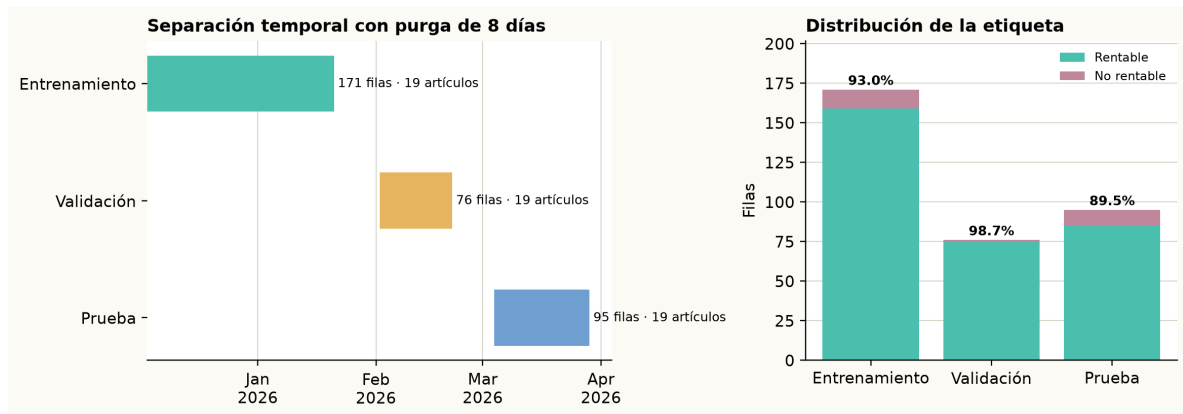


Figura 7.3: Corte temporal de marzo.

La selección compara las familias dummy, regresión logística, random forest e histogram gradient boosting. La configuración se elige en validación mediante precisión al umbral 0,85 con al menos tres señales. Después se calibra con sigmoide y se mide sobre el bloque de prueba. Además, se ejecuta una ablación de la regresión logística sin retardos, cambios, retornos ni medias móviles de uno, tres y siete días. Esta comparación responde a una pregunta concreta: si el histórico añade información o solamente complejidad. La precisión se presenta junto a la cobertura porque una clase mayoritaria puede hacer que el ROC-AUC parezca más concluyente de lo que permite la muestra [36].

La exploración no se limitó a escoger una familia. La Tabla 7.5 recoge el espacio de búsqueda empleado. El conjunto de prueba se dejó fuera de esta selección: sus valores se consultaron una vez después de fijar la configuración de validación.

La exploración inicial señala que la regresión logística sin características históricas puede separar mejor las dos clases que las alternativas probadas. También muestra que los retardos y medias móviles no deben darse por útiles sin contrastarlos. La siguiente

Familia	Configuraciones exploradas
Regresión logística	Regularización L2 con $C \in \{0,3,1,3\}$.
Random forest	Profundidad 10 o 18, 160 árboles y cinco muestras por hoja.
HGB	Tasa de aprendizaje 0,05 o 0,10, 160 iteraciones y 31 hojas.

Tabla 7.5: Configuraciones de marzo.

Configuración	ROC-AUC	Brier	Precisión @0,85	Señales
Logística con histórico	0,641	0,075	0,929	85
Logística sin histórico	0,760	0,072	0,970	67
Random forest con histórico	0,742	0,074	0,942	86
HGB con histórico	0,649	0,082	0,915	82

Tabla 7.6: Comparativa de marzo.

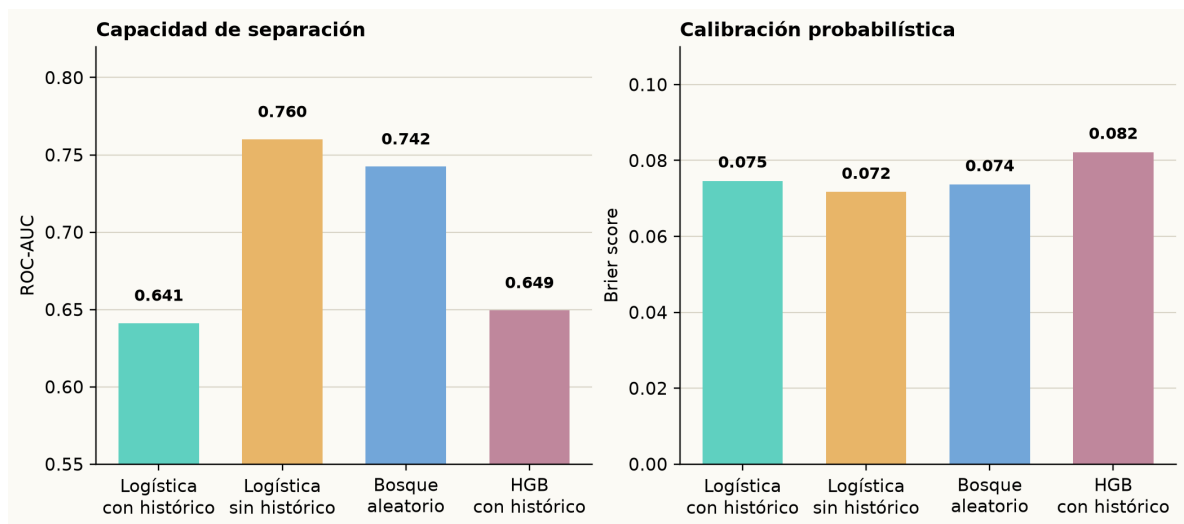


Figura 7.4: Modelos en marzo.

ablación vuelve a medir esa decisión junto con la aumentación, usando particiones por fecha para que una fecha completa no se divida entre lados de un pliegue.

7.4.6 Ablación de histórico y aumentación

La segunda exploración usa una matriz 2×2 con regresión logística. Se mantienen los mismos cortes temporales de marzo, el umbral de selección 0,85, tres pliegues temporales y calibración sigmoide. Se comparan dos decisiones de diseño: conservar o retirar retardos y medias móviles, y entrenar con o sin aumentación numérica. La aumentación se contrasta como hipótesis y no se presupone beneficiosa, ya que su efecto depende del dominio y del volumen de datos [40].

La aumentación se aplica únicamente al entrenamiento. Para cada variable numérica se genera una copia perturbada mediante

$$x'_k = x_k + \varepsilon_k, \quad \varepsilon_k \sim \mathcal{N}(0, 0,01 \cdot IQR(x_k)).$$

Las etiquetas, las variables categóricas y las fechas no cambian. Además, todas las filas de una misma fecha se mantienen dentro del mismo lado de cada pliegue temporal. De este modo la copia perturbada no puede aparecer en entrenamiento mientras su original está en validación. La validación y la prueba permanecen sin modificar.

Histórico	Aumentación	ROC-AUC	Brier	Precisión @0,85	Señales
Sí	No	0,631	0,076	0,931	87
No	No	0,716	0,076	0,952	62
Sí	Jitter	0,575	0,085	0,924	92
No	Jitter	0,613	0,084	0,919	86

Tabla 7.7: Ablación logística.

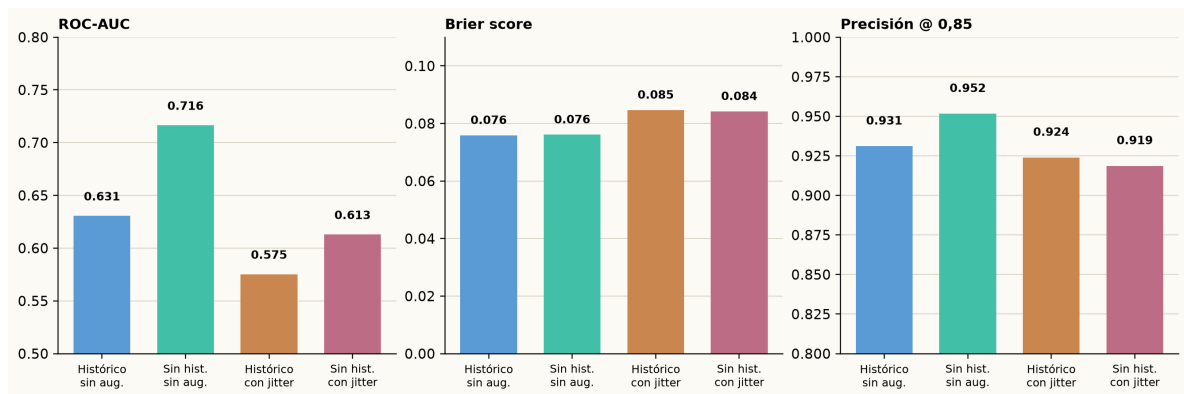


Figura 7.5: Histórico y aumentación.

La combinación sin histórico ni aumentación obtiene el mejor equilibrio entre separación y calibración. El jitter duplica el tamaño de entrenamiento de 171 a 342 filas, pero empeora ROC-AUC y Brier en ambas configuraciones. Por ello no se incorpora al modelo de referencia. Este resultado no prueba que la aumentación sea perjudicial en cualquier muestra. Indica que, con una colección corta y muy concentrada en

operaciones positivas, introducir pequeñas variaciones de precio añade más ruido que información.

La lectura debe mantenerse prudente. Los 19 artículos disponibles en el corte aparecen tanto en entrenamiento como en prueba y la etiqueta positiva representa el 89,5 % de la prueba. Una estrategia que marca muchos casos como favorables puede alcanzar precisión elevada por la propia distribución de la muestra. El corte de mayo confirma este problema: de 361 ejemplos de entrenamiento, 76 de validación y 95 de prueba, las dos últimas particiones contienen exclusivamente etiquetas positivas. En esas condiciones ROC-AUC no es una métrica definida y no se entrena un modelo para presentarlo como comparación válida. Este resultado establece una condición de continuación clara: ampliar la cobertura cruzada BUFF–Steam y equilibrar los eventos antes de convertir el modelo en recomendación operativa.

La Figura 7.6 complementa esta lectura. La precisión se mantiene cerca de la tasa positiva de la prueba mientras el umbral aumenta, pero el número de señales solo cae de forma apreciable a partir de 0,90. Por tanto, un umbral alto no convierte por sí mismo la señal en evidencia de selección robusta. La calibración y la precisión deben volver a evaluarse cuando haya una proporción relevante de operaciones no rentables.

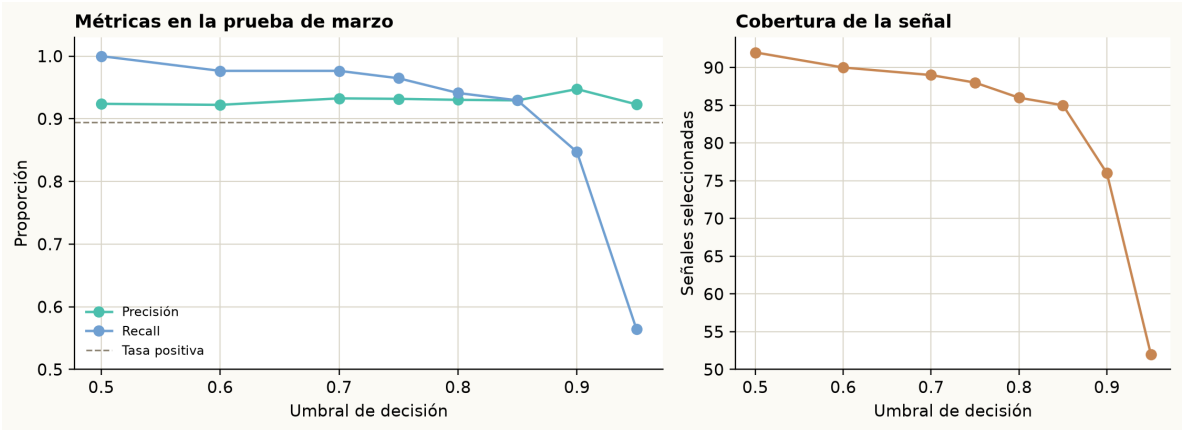


Figura 7.6: Métricas por umbral.

7.4.7 Comprobación del corte reciente

Con la base de datos conectada se repitió la construcción del dataset de BUFF a Steam el 11 de agosto de 2026. Se empleó la ventana desde diciembre de 2025, un horizonte de ocho días, una purga de ocho días y una prueba acotada a julio de 2026. El conjunto resultante conserva los 19 artículos ya presentes en el corte de marzo. El conjunto no permite entrenar otro clasificador y sirve para comprobar si la nueva captura ha aportado variedad suficiente de etiquetas.

Entrenamiento	Validación	Prueba
551 (95,6 %)	76 (100 %)	84 (98,8 %)

Tabla 7.8: Corte BUFF–Steam.

El resultado es deliberadamente incómodo, pero es importante. En validación y prueba falta una clase negativa, por lo que ROC-AUC, calibración y una comparación de políticas no son interpretables. Entrenar con esta muestra produciría una clasificación trivial y no aportaría evidencia sobre el sistema. La prioridad experimental pasa a ser mantener capturas recurrentes de más artículos, incluir oportunidades descartadas y conservar decisiones cerradas para que la recompensa MARL pueda distinguir compras acertadas, compras fallidas y decisiones de espera.

7.4.8 Evidencia de pruebas automatizadas

La suite completa se ejecutó el 14 de agosto de 2026 y obtuvo 282 pruebas superadas en 24,94 segundos. Las tres pruebas de integración utilizaron el `DATABASE_URL` PostgreSQL configurado en Supabase. Registran un snapshot de mercado, comprueban la actualización de una cotización y guardan una señal de oportunidad vinculada a un artículo. Cada una abre una transacción externa y la revierte al terminar, por lo que la comprobación no añade datos de prueba al histórico utilizado por los experimentos. Este registro de datos, configuración y resultados permite repetir las comprobaciones, práctica recomendada para hacer evaluables los resultados de aprendizaje automático [20].

Bloque	Pruebas	Alcance
Unitarias actuales	279	Economía, OCR, conectores, scraping, datasets, simulador, MARL y panel web.
Integración	3	Persistencia de snapshots, actualización de precios y señales en PostgreSQL.
Suite completa	282	Todas superadas en 24,94 segundos.

Tabla 7.9: Pruebas automatizadas.

7.4.9 Validación funcional de OCR

La evaluación de OCR disponible es funcional, no una medida de exactitud sobre un corpus etiquetado de capturas. Por ello no se comunica una tasa de reconocimiento que no se ha medido. Las pruebas comprueban que el parser convierte una observación de mercado en un contrato con artículo, calidad, plataforma, precio, moneda y volumen, que rechaza una confianza inferior a 0,5 y que el recorrido de imagen llama a carga, preprocesado y reconocimiento antes de persistir el resultado. El uso del motor se apoya en la arquitectura de Tesseract descrita por Smith [32]. La Tabla 7.10 delimita qué evidencia existe y qué queda pendiente.

Prueba	Caso ejecutado
Fixture de texto	Texto Steam y BUFF convertido a observaciones.
Umbral de confianza	Confianza 0,2 con mínimo configurado en 0,5.
Recorrido de imagen	Carga, preprocesado y reconocimiento con un runner controlado.

Tabla 7.10: Validación funcional de OCR.

La fixture confirma que el parser construye observaciones con los campos esperados a partir de texto de mercado. La prueba de confianza rechaza el valor 0,2 al exigir un mínimo de 0,5. El recorrido de imagen completa la secuencia de carga, preprocesado y reconocimiento y devuelve una confianza de 0,91 en el caso preparado.

Estas comprobaciones validan la integración, no la exactitud del OCR sobre capturas de interfaz. Por eso el siguiente experimento debe usar un conjunto etiquetado y comunicar por separado la exactitud de artículo, calidad, plataforma, precio y moneda. Hasta disponer de ese conjunto, el OCR se presenta como un conector validado y no como un sistema de reconocimiento cuantificado.

7.4.10 Uso operativo de los datos de compraventa

El dataset `trading_profit_v1` se generó desde `market_history_points` usando precios Steam/BUFF, comisiones y horizonte temporal. La primera versión produjo 2.331 ejemplos y 50 artículos, pero con desbalance extremo en validación y prueba. Por este motivo, sus métricas no se usan como evidencia de rendimiento operativo. Las correcciones posteriores del parser, la normalización y el backfill desembocan en `trading_profit_v2`, descrito en la Tabla 7.4.

El análisis posterior mostró que el histórico de BUFF empezó a estar mejor alineado tras corregir el parser y permitir backfill de puntos históricos. El conjunto actual ya permite entrenar y validar políticas en los tres bloques temporales, pero mantiene una distribución de etiquetas sesgada. Por ello, sus resultados se presentan con su tasa de operaciones rentables y no se interpretan como una garantía de rentabilidad futura.

A partir de esta limitación, la validación distingue entre aprendizaje temporal e inferencia operativa: Steam se usa como fuente principal para aprender el riesgo temporal de salida y BUFF queda como precio vivo de entrada en inferencia. El objetivo no es predecir el precio exacto de BUFF. La recomendación evalúa si un precio BUFF puntual conserva margen suficiente frente a una salida Steam estimada y sus comisiones.

7.4.11 Dificultades y pivotaciones

Durante el desarrollo se han producido varias decisiones de pivotaje. La primera aparece al formalizar el procedimiento manual: el valor principal estaba en extraer reglas económicas verificables y convertirlas en simulación reproducible. Esta decisión desplaza el trabajo desde una herramienta manual hacia un sistema auditable.

La segunda pivotación afecta a las fuentes de datos. El planteamiento inicial intentaba alinear históricos de Steam y BUFF para entrenar directamente sobre oportunidades de arbitraje entre plataformas. Las primeras pruebas mostraron dos problemas: falta de histórico alineado suficiente y riesgo operativo al consultar BUFF de forma reiterada. Por ello, el enfoque se ajusta: Steam se mantiene como fuente principal de aprendizaje temporal y BUFF se usa de forma puntual como precio vivo de entrada para evaluar flips concretos.

La tercera pivotación aparece en la parte multiagente. Antes de ampliar entrenamiento y episodios, se prioriza que el entorno, las recompensas, las restricciones y las métricas sean comprobables. Esta secuencia permite construir el entrenamiento sobre reglas económicas trazables, datos consistentes y comparaciones con baselines.

7.4.12 Controles funcionales del entorno MARL

La evaluación multiagente no utiliza una lista abstracta de acciones. Cada episodio recorre las observaciones del bloque de prueba de marzo en orden temporal. Scout indica si una oportunidad merece atención, Trader puede comprar una unidad, mantener o solicitar una venta y Portfolio aplica las restricciones de cartera. Una compra requiere el acuerdo de los tres roles. Una venta requiere que exista una posición desbloqueada del mismo artículo y que Portfolio la apruebe. De este modo, el episodio permite comprobar el intercambio entre una oportunidad local, las posiciones abiertas y el capital disponible para el conjunto de la cartera.

Elemento	Configuración comprobada
Observaciones	95 filas del bloque de prueba de marzo, en orden temporal.
Roles	Scout, Trader y Portfolio con observaciones locales.
Capital inicial	1.000 EUR en el recorrido de cartera de marzo.
Acciones	Compra con acuerdo de los tres roles y venta aprobada de posiciones desbloqueadas.
Estado de cartera	Efectivo, posiciones, exposición, bloqueo y drawdown.
Recompensa	Recompensa cooperativa al cerrar una operación, penalización por demora y señales individuales por rol.
Smoke PPO	Una iteración y 64 pasos mediante el adaptador PettingZoo-RLlib.

Tabla 7.11: Episodio MARL.

La diferencia entre el recorrido funcional y PPO debe mantenerse clara. La regla determinista se usa para verificar el ciclo de cartera completo. El smoke de PPO comprueba que el mismo entorno puede entregar observaciones y recompensas a RLlib. Estos controles no se utilizan para estimar el rendimiento de una política. Su finalidad es comprobar que el entorno que se usa después en el entrenamiento representa los ciclos de compra, bloqueo y venta definidos en el Capítulo 4.

7.4.13 Resultados de los controles funcionales

El entorno multiagente se ha comprobado con el bloque de marzo. Scout marca una oportunidad, Trader solicita una compra o una venta y Portfolio decide si la operación respeta saldo, exposición, volumen de intercambios y los límites de cartera. La recompensa cooperativa utilizada en estas ejecuciones se define en la Sección 4.5. La venta añade un resultado importante: el beneficio neto se registra al cerrar la posición, no solo como una estimación en el momento de comprar.

Ejecución	Finalidad	Pasos
Caso controlado	Compra, bloqueo de ocho días y venta de una posición.	2
Recorrido de cartera	Regla de margen positivo con compra, espera y venta.	95
PPO, una iteración	PPO multiagente mediante PettingZoo–RLlib.	64

Tabla 7.12: Ejecuciones MARL.

En el caso controlado se compró una unidad por 10,00 EUR en BUFF el 1 de enero de 2026. La venta permaneció inhabilitada hasta el 9 de enero. Al cerrarla en Steam por 14,00 EUR brutos, el simulador aplicó la comisión de venta y registró un ingreso neto de 12,18 EUR. La cartera pasó de 100,00 EUR a 102,18 EUR y el beneficio neto fue 2,18 EUR. Esta prueba confirma el cumplimiento del bloqueo y el registro de una venta, pero no evalúa una estrategia de selección.

El recorrido de cartera parte de 1.000 EUR y procesa 95 observaciones de marzo. La regla determinista compra cuando el margen neto observado es positivo y las restricciones lo permiten. Cuando encuentra una posición desbloqueada del mismo artículo, solicita su venta. Se ejecutaron 24 compras y 19 ventas, con un máximo de 11 posiciones simultáneas. Al terminar quedaron cinco posiciones abiertas, 306,60 EUR de capital bloqueado, 888,99 EUR de efectivo disponible y 195,59 EUR de beneficio neto obtenido al cerrar posiciones. El valor de cartera registrado fue 1.195,59 EUR.

Estas cifras verifican que el entorno conserva las posiciones abiertas, aplica límites y reutiliza efectivo después de una venta. No miden el rendimiento de PPO ni permiten estimar beneficio neto futuro: el corte contiene oportunidades preseleccionadas y está muy concentrado en márgenes positivos.

El comando PPO con RLlib completó una iteración y 64 pasos de entorno con el conjunto de entrenamiento. El estado central se expone al adaptador y hay una política por rol. La evaluación del checkpoint corto no ejecutó compras. Esta prueba valida la compatibilidad técnica, pero una única iteración no permite comparar una política aprendida con una regla de margen. La matriz de entrenamiento posterior repite semillas, episodios y configuraciones para comunicar la variabilidad del aprendizaje [41].

7.4.14 Matriz de entrenamiento MARL

La matriz actual entrena tres políticas por rol, Scout, Trader y Portfolio, a partir de las observaciones locales definidas en el entorno. Se emplea `trading_profit_v2`, 1.000 EUR de capital inicial, un máximo de 100 iteraciones, ocho episodios por iteración y parada temprana después de 20 iteraciones sin mejora en validación. Cada configuración se repite con las semillas 7, 19 y 31. En total se han completado 21 entrenamientos y 3.552 episodios, sin consultar el conjunto de prueba.

La configuración base usa un peso común de $g = 0,70$, un objetivo flexible de inversión del 50 %, tolerancia del 5 % y tasa de aprendizaje de 0,0003. A partir de ella se estudian el peso de la recompensa cooperativa, el objetivo de inversión, la tasa de aprendizaje y la retirada de la probabilidad supervisada. La Tabla 7.13 muestra el retorno del valor

final de cartera respecto del capital inicial, seleccionado por validación. Cada valor es la media de las tres semillas y el intervalo recoge la variabilidad observada.

Familia de configuración	Retorno medio	Intervalo entre semillas
Base	12,29 %	12,29–12,29 %
Recompensa cooperativa 50 %	12,29 %	12,29–12,29 %
Recompensa cooperativa 90 %	12,33 %	12,29–12,42 %
Objetivo de inversión 35 %	10,03 %	10,03–10,03 %
Objetivo de inversión 65 %	12,45 %	12,22–12,91 %
Tasa de aprendizaje 0,0001	12,29 %	12,29–12,29 %
Sin probabilidad supervisada	12,29 %	12,29–12,29 %

Tabla 7.13: Resultados de validación de la matriz MARL.

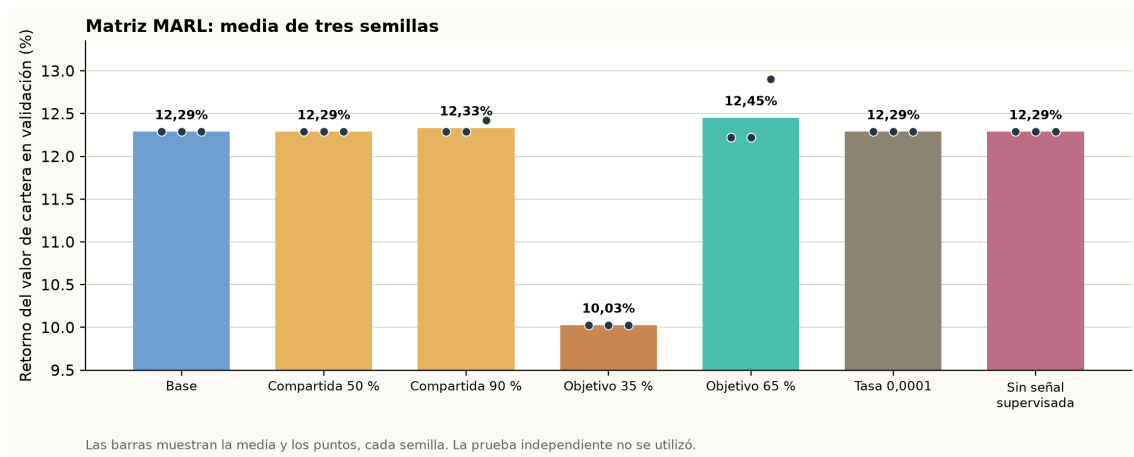


Figura 7.7: Resultados de validación de la matriz MARL. Cada punto representa una semilla.

El objetivo de inversión del 65 % obtiene la mayor media, con un 12,45 % frente al 12,29 % de la configuración base. Sin embargo, la diferencia es reducida y su mejor resultado, 12,91 %, procede de una sola semilla. Además, 19 de las 21 ejecuciones detuvieron el entrenamiento tras 20 iteraciones sin mejora. La matriz demuestra que el entrenamiento, la validación y la parada temprana se ejecutan de forma reproducible, pero no justifica todavía afirmar que una configuración supera de forma robusta a las demás.

7.5 Evaluación final reservada

La prueba de `trading_profit_v2` no se ha consultado durante la construcción de la matriz. Una vez fijada la configuración que se evaluará, debe ejecutarse una sola evaluación independiente con las tres semillas elegidas. Esta evaluación comunicará el retorno de cartera, el beneficio neto, las operaciones cerradas, las posiciones abiertas y las restricciones rechazadas. El resultado se presentará como media e intervalo entre semillas, no como el mejor valor individual.

La comparación final debe incluir baselines homogéneos: mantener efectivo, comprar cuando el margen observado es positivo y un agente único con las mismas restricciones. La matriz ya incorpora una ablación sin probabilidad supervisada. Las siguientes extensiones son retirar el agente Portfolio o modificar los pesos de recompensa, siempre seleccionando los parámetros con validación y dejando la prueba aislada. Estas comparaciones permitirán distinguir si una diferencia proviene de la arquitectura multiagente, de la señal supervisada, de una regla de cartera o de la estructura del simulador.

7.6 Discusión

La discusión relaciona los resultados con el alcance del trabajo. Se identifican las condiciones que limitan su interpretación, las partes que ya funcionan de forma integrada y los datos que faltan para una evaluación más sólida.

7.6.1 Amenazas a la validez

La principal amenaza interna es el desbalance de etiquetas: la prueba de marzo contiene 85 casos rentables de 95 y los bloques posteriores llegan a tener una sola clase. La segunda es el solapamiento de artículos entre entrenamiento y prueba. La división por fecha protege frente al uso de futuro, pero no responde a la pregunta más exigente de generalizar a activos virtuales no presentes en el entrenamiento. La tercera amenaza es operativa: el precio BUFF se usa como entrada puntual y la salida Steam se estima con histórico y comisiones, por lo que la sincronización exacta de ambos mercados requiere más capturas. Además, los experimentos simulan operaciones con datos observados. No envían órdenes reales, no transfieren artículos entre plataformas ni utilizan capital real, por lo que no permiten garantizar la ejecución futura de una operación.

También hay límites de medición. El OCR está validado a nivel funcional, pero aún no dispone de un corpus de imágenes etiquetado. El entorno MARL ya incluye compra, bloqueo, venta y una matriz de políticas entrenadas, pero las configuraciones actuales presentan diferencias pequeñas en validación y todavía no se han contrastado en la prueba independiente. Estas limitaciones no invalidan la arquitectura, pero acotan las conclusiones a viabilidad técnica y evidencia experimental inicial.

7.6.2 Casos favorables

El trabajo realizado muestra que el sistema ya puede integrar fuentes, construir datasets, entrenar modelos supervisados, calibrar probabilidades, simular reglas económicas y ejecutar un smoke multiagente. La separación entre adquisición, predicción, simula-

ción y decisión ha resultado útil porque permite avanzar por partes y detectar bloqueos concretos sin rehacer toda la arquitectura.

7.6.3 Limitaciones observadas

La principal limitación actual es la disponibilidad de histórico alineado entre Steam y BUFF. Aunque el conjunto actual contiene ejemplos no rentables en los tres bloques, validación y prueba siguen concentradas en operaciones rentables. También existe riesgo de sobreajuste en variables categóricas de alta cardinalidad y en señales de precisión alta con pocos casos. Las reglas de las plataformas, sus comisiones y la actividad de intercambio también pueden cambiar con el tiempo, por lo que los resultados deben interpretarse según las condiciones observadas en cada captura. Por último, el entorno MARL necesita la evaluación independiente y los baselines descritos antes de poder concluir si la arquitectura multiagente mejora a una estrategia simple.

7.6.4 Lectura de los resultados

Los resultados confirman la viabilidad técnica de la integración y la simulación, así como la ejecución reproducible de una matriz de entrenamiento multiagente. La comparación definitiva del rendimiento económico entre políticas requiere ahora cerrar la evaluación independiente, ampliar los datos y contrastar baselines homogéneos.

Conclusiones

Este capítulo resume las aportaciones del trabajo, revisa el grado de cumplimiento de los objetivos y delimita los pasos necesarios para ampliar la validación experimental y el entrenamiento multiagente.

8.1 Conclusiones principales

El desarrollo realizado hasta el momento permite concluir que el problema no puede reducirse a una predicción aislada de subida o bajada de precio. En mercados como el de *Counter-Strike 2*, la decisión depende de fricciones muy concretas: comisiones, conversión de moneda, volumen de intercambios, *trade hold*, capital bloqueado y riesgo de cartera. Por ello, la arquitectura propuesta necesita combinar adquisición de datos, simulación económica y decisión multiagente, siguiendo la disciplina de backtesting y control de fricciones propia de los entornos de trading con aprendizaje automático [17].

También se ha comprobado que la separación por capas facilita el avance del proyecto. Los agentes extractores se encargan de observar el mercado, el modelo supervisado produce señales probabilísticas, el simulador aplica reglas económicas y los agentes MARL operan dentro de un entorno controlado. Esta división reduce el acoplamiento y permite validar cada parte por separado.

La contribución principal del desarrollo realizado no es afirmar que el sistema gane dinero, sino construir una base reproducible para estudiar si el sistema puede seleccionar operaciones rentables bajo las restricciones del mercado. El proyecto ya dispone de una arquitectura modular, un flujo de adquisición, un modelo de persistencia, un simulador económico, datasets derivados, modelos supervisados preliminares y un entorno multiagente funcional para smoke tests.

Una conclusión metodológica relevante es que el proyecto ha evolucionado a partir de las dificultades encontradas. La falta de histórico Steam/BUFF suficientemente alineado, el riesgo de scraping reiterado y el desbalance de los primeros datasets de trading han llevado a separar entrenamiento e inferencia operativa. La separación entre entrenamiento e inferencia no cambia el objetivo del sistema, pero sí mejora su viabilidad: el aprendizaje temporal se apoya en Steam y la comparación con BUFF se reserva para evaluar oportunidades concretas.

8.2 Cumplimiento de objetivos

Se ha avanzado en la mayoría de objetivos técnicos: existe un monorepo modular, un modelo de datos operativo, pipelines de adquisición, análisis del procedimiento manual inicial, construcción de datasets, entrenamiento supervisado, simulador de cartera y un entorno MARL funcional para pruebas cortas. También se han definido agentes Scout, Trader y Portfolio, junto con restricciones de riesgo y métricas de evaluación.

El cumplimiento experimental todavía es parcial. Los resultados supervisados muestran que el pipeline funciona y que algunos modelos producen señales de alta precisión en umbrales exigentes, pero el número de señales es limitado. El dataset operativo de trading Steam/BUFF está construido, aunque sus cortes presentan desbalance extremo en validación y test. El entorno MARL ya ha completado ciclos de compra, bloqueo y venta con varias posiciones, pero no constituye todavía una evaluación comparativa completa. Las pruebas automatizadas y los smoke tests de rendimiento forman parte del capítulo experimental porque verifican que las métricas obtenidas proceden de componentes comprobados.

8.3 Limitaciones

La limitación más importante es la escasez de histórico alineado entre plataformas para algunas direcciones de trading. El dataset más cercano al objetivo operativo carece de suficientes ejemplos negativos en validación y test, por lo que no permite defender todavía una mejora robusta. Además, algunos resultados supervisados tienen alta precisión en umbrales altos, pero con pocas señales, lo que obliga a interpretarlos con prudencia. Por ello, se separa el aprendizaje temporal basado en Steam del uso puntual de BUFF como precio vivo de entrada para flips.

Otra limitación es que el smoke multiagente demuestra que el flujo ejecuta, pero no que el sistema haya aprendido una política superior. Para llegar a esa conclusión se necesita una evaluación reproducible con más episodios, más datos, baselines justos y métricas de riesgo.

También existe una limitación propia del dominio. Los activos de CS2 dependen de plataformas externas, reglas de intercambio, sesiones, cambios visuales y restricciones que pueden variar. Por tanto, cualquier sistema automatizado debe mantener trazabilidad, parámetros versionados y capacidad de revisar errores de fuente.

8.4 Trabajo futuro

Las líneas inmediatas de trabajo son validar un modelo Steam-only de salida a ocho días, usar BUFF como precio vivo de entrada en inferencia de flip, comparar contra baselines simples y entrenar MARL con una configuración multiagente completa. Después será necesario estudiar ablations, generar gráficas de resultados y cerrar una discusión experimental sobre cuándo la arquitectura multiagente aporta valor y cuándo no.

Los experimentos finales deberán reportar beneficio neto, saldo final, drawdown, capital bloqueado, número de operaciones, posiciones abiertas y rechazos por riesgo. Además, deberán comparar la arquitectura completa contra versiones simplificadas: sin probabilidad supervisada, sin agente Portfolio y con reglas simples de spread. Solo con

esa comparación será posible defender si la descentralización multiagente aporta una mejora frente a estrategias más sencillas, siguiendo la lógica comparativa habitual en MARL y entrenamiento cooperativo [14, 15].

Bibliografía

- [1] Vili Lehdonvirta. Virtual item sales as a revenue model: Identifying attributes that drive purchase decisions. *Electronic Commerce Research*, 9(1):97–113, 2009.
- [2] Yue Guo and Stuart J. Barnes. Virtual item purchase behavior in virtual worlds: An exploratory investigation. *Electronic Commerce Research*, 9(1–2):77–96, 2009.
- [3] Valve. Steam community market faq. <https://help.steampowered.com/en/faqs/view/61F0-72B7-9A18-C70B>. Consultado en agosto de 2026.
- [4] Roblox. The marketplace. <https://en.help.roblox.com/hc/en-us/articles/203313300-The-Marketplace>. Consultado en agosto de 2026.
- [5] Blizzard Entertainment. Using a wow token. <https://us.support.blizzard.com/en/article/31218>. Consultado en agosto de 2026.
- [6] CCP Games. Buy and sell orders. <https://support.eveonline.com/hc/en-us/articles/203218932-Buy-and-Sell-Orders>. Consultado en agosto de 2026.
- [7] BUFF. Buff market. <https://buff.163.com/market/csgo>. Consultado en agosto de 2026.
- [8] Skinport. Items | skinport rest api. <https://docs.skinport.com/items>. Consultado en agosto de 2026.
- [9] BitSkins. Bitskins market. <https://bitskins.com/market/skin/>. Consultado en agosto de 2026.
- [10] DMarket. Dmarket marketplace. <https://dmarket.com/marketplace>. Consultado en agosto de 2026.
- [11] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.
- [12] Michael Wooldridge. *An Introduction to MultiAgent Systems*. Wiley, 2 edition, 2009.
- [13] Lucian Buşoniu, Robert Babuška, and Bart De Schutter. A comprehensive survey of multiagent reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 38(2):156–172, 2008.
- [14] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems*, 2017.
- [15] Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative, multi-agent games. In *Advances in Neural Information Processing Systems*, 2022.

- [16] Harry Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.
- [17] Xiao-Yang Liu, Hongyang Yang, Jiechao Gao, and Christina Dan Wang. FinRL: Deep reinforcement learning framework to automate trading in quantitative finance. In *Proceedings of the Second ACM International Conference on AI in Finance*, 2021.
- [18] Valve. Counter-strike 2. https://store.steampowered.com/app/730/CounterStrike_2/. Consultado en agosto de 2026.
- [19] SteamDT. Steamdt: Cs2 market overview. <https://www.steamdt.com/en>. Consultado en agosto de 2026.
- [20] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d’Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research. *Journal of Machine Learning Research*, 22(164):1–20, 2021.
- [21] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998.
- [22] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12(85):2825–2830, 2011.
- [23] Justin K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis Santos, Rodrigo Perez, Caroline Horsch, Clemens Diefendahl, Niall L. Williams, Yashas Lokesh, Praveen Ravi Hines, and Niall Ravi. Pettingzoo: Gym for multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, 2021.
- [24] Eric Liang, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph E. Gonzalez, Michael I. Jordan, and Ion Stoica. Rllib: Abstractions for distributed reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning*, 2018.
- [25] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pages 8024–8035, 2019.
- [26] pytest-dev team. pytest documentation. <https://docs.pytest.org/en/stable/contents.html>, 2026. Consultado el 27 de agosto de 2026.
- [27] Astral Software Inc. Ruff documentation. <https://docs.astral.sh/ruff/>, 2026. Consultado el 27 de agosto de 2026.

- [28] Mypy project. mypy documentation. <https://mypy.readthedocs.io/>, 2026. Consultado el 27 de agosto de 2026.
- [29] Microsoft. Playwright: Browser automation documentation. <https://playwright.dev/docs/api/class-playwright>, 2026. Consultado el 25 de agosto de 2026.
- [30] Gary Bradski. The OpenCV library. *Dr. Dobb's Journal of Software Tools*, 2000.
- [31] Karel Zuiderveld. Contrast limited adaptive histogram equalization. In Paul S. Heckbert, editor, *Graphics Gems IV*, pages 474–485. Morgan Kaufmann, 1994.
- [32] Ray Smith. An overview of the Tesseract OCR engine. In *Ninth International Conference on Document Analysis and Recognition*, volume 2, pages 629–633, 2007.
- [33] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [34] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.
- [35] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. <https://arxiv.org/abs/1707.06347>, 2017.
- [36] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015.
- [37] Alexandru Niculescu-Mizil and Rich Caruana. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning*, pages 625–632, 2005.
- [38] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pages 1321–1330. PMLR, 2017.
- [39] Christoph Bergmeir, Rob J. Hyndman, and Bonsoo Koo. A note on the validity of cross-validation for evaluating autoregressive time series prediction. *Computational Statistics & Data Analysis*, 120:70–83, 2018.
- [40] Brian Kenji Iwana and Seiichi Uchida. An empirical survey of data augmentation for time series classification with neural networks. *PLOS ONE*, 16(7):e0254841, 2021.
- [41] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C. Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, volume 34, pages 29304–29320, 2021.

Anexos

Los anexos reúnen detalles operativos que complementan la explicación principal: estructura del repositorio, configuración del entorno, comandos reproducibles y parámetros de entrenamiento.

A.1 Estructura del repositorio

El código fuente completo se encuentra en el repositorio público del proyecto:

<https://github.com/STOSKR/TFM-CSFlipper>

La Figura A.1 resume la organización. Se muestran solo los directorios que ayudan a localizar cada parte del sistema, mientras que los ficheros y resultados generados se consultan directamente en el repositorio.

TFM-CSFlipper/	
apps/	comandos, extractores e interfaz local
packages/	lógica reutilizable del sistema
tests/	pruebas unitarias y de integración
data/	datasets y resultados derivados
docs/	documentación técnica
TFM/	memoria, figuras y bibliografía

Figura A.1: Estructura del repositorio.

Los directorios principales cumplen estas funciones:

- **apps**: contiene los extractores, los comandos de consola y la interfaz local.
- **packages**: agrupa los contratos, la persistencia, la predicción, el simulador, el entorno MARL y el procesamiento visual.
- **tests**: reúne las pruebas unitarias y de integración que verifican reglas económicas, conectores y flujos principales.
- **data**: conserva datasets derivados, informes y resultados que no forman parte del código fuente.
- **docs**: recoge documentación técnica y decisiones de desarrollo.
- **TFM**: contiene el código L^AT_EX, las figuras y la bibliografía de esta memoria.

A.2 Configuración del entorno

El entorno se define en `pyproject.toml`. El proyecto usa Python, Supabase/Postgres, Pytest, Ruff, Mypy y dependencias opcionales para MARL como Ray/RLlib, PettingZoo, Gymnasium y PyTorch. Para desarrollo local existe un `docker-compose.yml` con Postgres.

Comandos representativos:

```
python -m pytest tests/unit
python -m ruff check
python -m mypy apps packages tests/unit
python -m apps.cli.auto_scrape_loop --once
python -m apps.cli.render_stream_scrape
python -m apps.cli.web_dashboard_server
```

A.3 Comandos de datasets y modelos

Los comandos de datos y modelos separan adquisición, construcción de dataset, entrenamiento e inferencia. Esta separación permite repetir una fase sin rehacer todo el flujo.

```
python -m apps.cli.build_supervised_dataset
python -m apps.cli.analyze_supervised_coverage
python -m apps.cli.train_supervised_model
python -m apps.cli.build_trading_dataset
python -m apps.cli.retrain_trading_model
python -m apps.cli.score_live_opportunities
```

A.4 Parámetros de entrenamiento

Los experimentos supervisados usan splits temporales, calibración de probabilidades y selección por precisión operativa en umbrales altos. En los smoke tests se han comparado modelos dummy, regresión logística, random forest e histogram gradient boosting. El entrenamiento MARL usa RLlib con un entorno multiagente y tres políticas: Scout, Trader y Portfolio.

A.5 Modelo de datos

El modelo operativo actual gira alrededor de dos tablas principales: `market_items` y `market_history_points`. La primera representa artículos identificados por sus atributos y su estado actual por plataforma. La segunda almacena series temporales en formato largo, con una fila por artículo, plataforma, fecha y métrica. La separación entre `market_items` y `market_history_points` evita mezclar métricas distintas y permite añadir nuevas señales sin alterar la forma general del histórico.

El esquema canónico previsto añade entidades como `assets`, `platforms`, `market_observations`, `predictions`, `agent_actions`, `investment_decisions` y

Bloque	Parámetros relevantes
Supervisado	Split temporal, umbral operativo, calibración, modelo candidato y columnas excluidas por leakage.
Trading	Dirección de arbitraje, horizonte, fees, tipo de cambio, ventana histórica y target de beneficio neto.
Simulación	Saldo inicial, tamaño máximo de posición, <i>trade hold</i> , comisiones y límites de exposición.
MARL	Número de episodios, políticas Scout/Trader/Portfolio, algoritmo PPO y métricas de cartera.

Tabla A.1: Configuración experimental.

`simulated_positions`. El objetivo de este esquema es mejorar trazabilidad, auditoría y separación entre observación, predicción y decisión.

A.6 Resultados adicionales

Los resultados intermedios se guardan en `model-runs` y los datasets derivados en `data/datasets`. Estas carpetas permiten conservar reportes de cobertura, exploración de características, métricas de entrenamiento, umbrales seleccionados y salidas de smoke tests sin mezclar los artefactos pesados con el código principal.

Para que un experimento aporte evidencia reproducible, debe registrar como mínimo: configuración usada, periodo temporal, número de activos, tamaño de cada split, tasa de positivos, modelo o política evaluada, métricas principales y limitaciones observadas.